

Laboratorio de Kubernetes con Monitorización y Seguridad

Proyecto integrador

Álvaro Cabello Aguilar

Curso 2025/2026

Índice

1	Fase 1 – Preparación de infraestructura	2
1.1	Objetivo	2
1.2	Requisitos de la infraestructura	2
1.3	Instalación de paquetería en el <i>host</i>	2
1.4	Definición de la red privada	3
1.5	Vagrantfile	4
1.6	Provisioners de fase 0 (<i>config.sh</i> y <i>ssh-setup.sh</i>)	5
1.6.1	<i>config.sh</i>	5
1.6.2	<i>ssh-setup.sh</i>	5
1.7	Trigger <i>ssh-config.sh</i>	6
1.8	Discos persistentes	6
1.8.1	Decisión arquitectónica	6
1.8.2	<i>create-disks.sh</i> (<i>trigger</i> en el host)	6
1.8.3	<i>attach-disks.sh</i> (<i>trigger</i> en el host)	6
1.9	Acceso gráfico a las VMs	7
1.10	Resumen del estado tras Fase 1	7
2	Fase 2 – Despliegue del clúster Kubernetes	7
2.1	Objetivo	7
2.2	Por qué Kubespray	7
2.3	Estructura del <i>script</i> <i>setup-kubespray.sh</i>	8
2.3.1	Fase 1: Clonado de Kubespray	8
2.3.2	Fase 2: Entorno virtual de Python	8
2.3.3	Fase 3: Generación del inventario	8
2.3.4	Fase 4: Configuración de <i>group_vars</i>	9
2.3.5	Fase 5: Ejecución del <i>playbook</i>	9
2.3.6	Fase 6: Instalación de <i>kubeconfig</i>	9
2.3.7	Fase 7: Creación de <i>namespaces</i>	9
2.4	Validación post-despliegue	10
2.5	Lecciones y <i>gotchas</i> encontradas	10
2.5.1	Re-exec a usuario <i>master</i> con <i>chmod a+r</i>	10
2.5.2	Per-host <i>ansible_user</i>	10
2.5.3	Inter-node SSH <i>passwordless</i>	10
2.5.4	Idempotencia del clúster	10
3	Fase 3 – <i>Stack</i> de monitorización	10
3.1	Objetivo	10
3.2	Arquitectura	11
3.3	Volúmenes persistentes vía <i>hostPath</i>	11
3.4	Grafana	12
3.4.1	Despliegue	12
3.4.2	Provisioning declarativo de <i>datasources</i>	12
3.4.3	Provisioning declarativo de <i>dashboards</i>	13
3.5	Prometheus	13

3.6	Loki	14
3.6.1	Modo <code>SingleBinary</code>	14
3.6.2	Configuración base	14
3.6.3	<i>Sizing</i> bajo <i>pen-test flood</i>	14
3.6.4	Tamaño máximo de mensaje gRPC	15
3.6.5	Límites de <i>query</i>	15
3.7	Promtail	15
3.8	Convención de <i>labels</i> en LogQL	16
3.9	El <i>script</i> <code>setup-lab.sh</code>	16
4	Fase 4 – <i>Stack</i> de seguridad y CMS vulnerables	16
4.1	Objetivo	16
4.2	Arquitectura	16
4.3	Concentración en <code>worker1</code>	17
4.4	Falco	17
4.4.1	Driver <code>modern_ebpf</code>	17
4.4.2	Salida JSON a <code>stdout</code>	17
4.4.3	El parámetro <code>base_syscalls.repair</code>	17
4.4.4	Regla custom: “Lab — Shell en container”	18
4.4.5	<code>customRules</code> y <code>rules_files</code> : la trampa	18
4.5	Suricata	18
4.5.1	<code>hostNetwork</code> y <i>capabilities</i>	19
4.5.2	Captura con <code>af-packet</code>	19
4.5.3	Variables de red: <code>HOME_NET</code> y <code>HTTP_PORTS</code>	19
4.5.4	El <i>init container</i> : descarga del <code>ruleset</code> ET Open	20
4.5.5	El <i>sidecar</i> <code>tail -F eve.json</code>	20
4.5.6	Coherencia config: <code>stats</code>	21
4.6	DVWA — <i>Damn Vulnerable Web Application</i>	21
4.6.1	Imagen <code>vulnerables/web-dvwa</code>	21
4.6.2	Sin persistencia (decisión consciente)	21
4.6.3	Probe <code>/login.php</code>	21
4.6.4	<code>strategy</code> : <code>Recreate</code>	21
4.7	WebGoat	22
4.7.1	Imagen oficial	22
4.7.2	Variables de entorno críticas	22
4.7.3	URL <i>case-sensitive</i> : <code>/WebGoat</code>	22
4.8	Juice Shop	22
4.8.1	Despliegue mínimo	22
4.8.2	Sin login inicial	22
4.9	El <i>script</i> <code>setup-security.sh</code>	23
4.10	Lecciones transversales	23
4.10.1	Charts custom: límite a Helm <i>upstream</i>	23
4.10.2	Imágenes con <i>defaults</i> hostiles a <code>NodePort</code>	23
4.10.3	Detectores silenciosamente rotos	23
4.11	Estado tras Fase 4	24
A	Scripts	24
A.1	<code>network-check.sh</code>	24
A.2	<code>create-disks.sh</code>	24
A.3	<code>config.sh</code>	25
A.4	<code>ssh-setup.sh</code>	25
A.5	<code>ssh-config.sh</code>	26
A.6	<code>attach-disks.sh</code>	26
A.7	<code>setup-kubespray.sh</code>	28
A.8	<code>setup-lab.sh</code>	30
A.9	<code>setup-security.sh</code>	32

1 Fase 1 – Preparación de infraestructura

1.1 Objetivo

El laboratorio consiste en tres máquinas virtuales Ubuntu 22.04 orquestadas con Vagrant + libvirt sobre una red privada propia (**red-prueba**). El despliegue es totalmente reproducible mediante *infraestructura como código*: un único **vagrant up** desde el host crea la red, los discos persistentes, las VMs, los usuarios personalizados y deja las máquinas listas para que cualquiera de los *provisioners* de fases posteriores (Kubespray, monitoring, seguridad) pueda ejecutarse a demanda.

1.2 Requisitos de la infraestructura

A continuación se muestran las especificaciones técnicas asignadas a cada nodo en el **Vagrantfile**:

Infraestructura nodo test-master	
Componente	Especificación
vCPU	2 núcleos
RAM	4096 MB
Disco principal	40 GB (caja Vagrant)
Disco persistente	40 GB qcow2 (/dev/vdb)
Red privada	red-prueba (NAT, virbr2)
IP	10.0.1.10
Sistema Operativo	Ubuntu 22.04 LTS
Usuario personalizado	master

Infraestructura nodo test-worker1	
Componente	Especificación
vCPU	2 núcleos
RAM	3072 MB
Disco principal	40 GB (caja Vagrant)
Disco persistente	20 GB qcow2 (/dev/vdb)
Red privada	red-prueba (NAT, virbr2)
IP	10.0.1.11
Sistema Operativo	Ubuntu 22.04 LTS
Usuario personalizado	worker1

Infraestructura nodo test-worker2	
Componente	Especificación
vCPU	1 núcleo
RAM	2048 MB
Disco principal	40 GB (caja Vagrant)
Disco persistente	20 GB qcow2 (/dev/vdb)
Red privada	red-prueba (NAT, virbr2)
IP	10.0.1.12
Sistema Operativo	Ubuntu 22.04 LTS
Usuario personalizado	worker2

El reparto de RAM no es uniforme deliberadamente: **test-worker1** aloja los tres CMS vulnerables (DVWA, WebGoat, Juice Shop) junto a Suricata, mientras que **test-worker2** concentra todo el *stack* de monitorización (Grafana, Prometheus, Loki) sobre **hostPath** local. **test-master** solo aloja el *control-plane* de Kubernetes y etcd.

1.3 Instalación de paquetería en el *host*

A continuación se listan los paquetes¹ necesarios en la máquina anfitriona:

- **Libvirt y QEMU**

¹El *host* desde el que se desarrolla el proyecto es Arch Linux con **pacman** como gestor de paquetes. La reproducción es posible en otras distribuciones cambiando el gestor; los nombres de los paquetes son comunes en la mayoría de distros mainstream.

```
1 sudo pacman -S qemu-full virt-manager virt-viewer spice-gtk virt-install libvirt
2 sudo pacman -S --needed dnsmasq vde2 iproute2 iptables-nft openbsd-netcat
```

- **Activación de los servicios**

```
1 sudo systemctl enable --now libvirtd.service
2 sudo systemctl enable --now virtlogd.service
```

- **Grupo libvirt:** necesario para ejecutar `virsh` sin privilegios de superusuario:

```
1 sudo gpasswd -a $USER libvirt
2 newgrp libvirt
```

- **Vagrant** (vía AUR con `yay`):

```
1 yay -S vagrant
2 vagrant plugin install vagrant-libvirt
```

- **Herramientas auxiliares** usadas por los *scripts* del proyecto:

- `qemu-img`: creación de discos `qcow2` (incluido en `qemu-full`).
- `setfacl`: gestión de ACL para permisos del usuario `libvirt-qemu` (paquete `acl`).
- `virsh`: gestión de dominios y redes de libvirt.

1.4 Definición de la red privada

La red privada `red-prueba` es gestionada por libvirt y su definición vive en `network/red-prueba.xml`, dentro del propio repositorio. La automatización es responsabilidad de `scripts/network-check.sh`, que se ejecuta como *trigger before* `:up` antes de levantar la primera VM.

```
1 <network>
2   <name>red-prueba</name>
3   <forward mode='nat' />
4   <bridge name='virbr2' stp='on' delay='0' />
5   <ip address='10.0.1.1' netmask='255.255.255.0'>
6     <dhcp>
7       <range start='10.0.1.2' end='10.0.1.254' />
8       <host mac='52:54:00:00:01:10' name='test_master' ip='10.0.1.10' />
9       <host mac='52:54:00:00:01:11' name='test_worker1' ip='10.0.1.11' />
10      <host mac='52:54:00:00:01:12' name='test_worker2' ip='10.0.1.12' />
11    </dhcp>
12  </ip>
13 </network>
```

La asignación de IP estática por reserva DHCP *con MAC fija* es obligatoria: si no se fijan las MAC, Vagrant crearía una segunda red automática (*management*) y se perdería la trazabilidad de las IPs reales. La asignación final queda:

Nodo	IP	MAC
test-master	10.0.1.10	52:54:00:00:01:10
test-worker1	10.0.1.11	52:54:00:00:01:11
test-worker2	10.0.1.12	52:54:00:00:01:12

Comprobación previa al up. El *script* `network-check.sh` no solo verifica la existencia de la red: compara la configuración *activa* en libvirt con el XML del repositorio (ignorando UUIDs y MACs auto-generados) y, si difieren, destruye y redefine la red. Esto evita un fallo silencioso muy frecuente: tener una red `red-prueba` *antigua* con un *range* o un MAC distinto que no *matchea* la del `Vagrantfile` y descubrir solo más tarde que las VMs reciben IPs aleatorias.

1.5 Vagrantfile

El despliegue se realiza con Vagrant + el provider `vagrant-libvirt`. Se descartó Terraform por estar enfocado a infraestructura *cloud* y no ofrecer integración directa con libvirt local sin un *provider* de la comunidad poco mantenido. A continuación, el Vagrantfile completo:

```
1 ENV['VAGRANT_NO_PARALLEL'] = 'yes'
2
3 Vagrant.configure("2") do |config|
4   config.vm.box = "generic/ubuntu2204"
5
6   # Synced folder: rsync (el unico modo fiable con vagrant-libvirt sin NFS).
7   # Excluimos .git/.vagrant/disks para no rsynchronizar GB de qcow2.
8   config.vm.synced_folder ".", "/vagrant", type: "rsync",
9     rsync__exclude: [".git/", ".vagrant/", "disks/"]
10
11   nodes = {
12     "test-master" => { "ip" => "10.0.1.10", "mac" => "52:54:00:00:01:10", "user" => "
13       master" },
14     "test-worker1" => { "ip" => "10.0.1.11", "mac" => "52:54:00:00:01:11", "user" => "
15       worker1" },
16     "test-worker2" => { "ip" => "10.0.1.12", "mac" => "52:54:00:00:01:12", "user" => "
17       worker2" }
18   }
19
20   nodes.each do |name, info|
21     config.vm.define name do |node|
22       node.vm.hostname = name
23
24       node.vm.network "private_network",
25         ip: info["ip"],
26         netmask: "255.255.255.0",
27         mac: info["mac"],
28         libvirt__network_name: "red-prueba",
29         libvirt__forward_mode: "nat",
30         libvirt__always_destroy: false
31
32       node.vm.provider :libvirt do |libvirt|
33         libvirt.memory = case name
34           when "test-master" then 4096
35           when "test-worker1" then 3072
36           else 2048
37         end
38         libvirt.cpus = (name == "test-master" || name == "test-worker1") ? 2 : 1
39         libvirt.mgmt_attach = false # sin red de management adicional
40       end
41
42       # Provisioners encadenados en todas las VMs:
43       node.vm.provision "shell" do |s|
44         s.path = "scripts/config.sh" # paquetes base (vim, python3, git...)
45       end
46       node.vm.provision "shell" do |s|
47         s.path = "scripts/ssh-setup.sh" # crea el usuario custom + claves inter-nodo
48         s.args = [info["user"]]
49       end
50
51       # Provisioners "run: never" solo en master, disparados manualmente:
52       if name == "test-master"
53         node.vm.provision "setup-kubespray", type: "shell", run: "never" do |s|
54           s.path = "scripts/setup-kubespray.sh"
55           s.privileged = false
56         end
57         node.vm.provision "setup-lab", type: "shell", run: "never" do |s|
58           s.path = "scripts/setup-lab.sh"
59           s.privileged = false
60         end
61         node.vm.provision "setup-security", type: "shell", run: "never" do |s|
62           s.path = "scripts/setup-security.sh"
63           s.privileged = false
64         end
65       end
66     end
67   end
68 end
```

```

65
66 # Triggers del host (fuera del ciclo de provisioners):
67 config.trigger.before :up, only_on: "test-master" do |trigger|
68   trigger.name = "[+] Comprobacion de la red puente"
69   trigger.run = { path: "scripts/network-check.sh" }
70 end
71 config.trigger.before :up, only_on: "test-master" do |trigger|
72   trigger.name = "[+] Comprobacion del almacenamiento: discos qcow2"
73   trigger.run = { path: "scripts/create-disks.sh" }
74 end
75 config.trigger.after :up, only_on: "test-worker2" do |trigger|
76   trigger.name = "[+] Configuracion SSH las VMs"
77   trigger.run = { path: "scripts/ssh-config.sh" }
78 end
79 config.trigger.after :up, only_on: "test-worker2" do |trigger|
80   trigger.name = "[+] Conexion y montaje persistente de los discos"
81   trigger.run = { path: "scripts/attach-disks.sh" }
82 end
83 end

```

Despliegue secuencial. La variable de entorno `VAGRANT_NO_PARALLEL = 'yes'` fuerza el arranque secuencial de las VMs. Esto evita condiciones de carrera entre *triggers* y *provisioners*: por ejemplo, `ssh-config.sh` se ejecuta como *trigger after :up, only_on: test-worker2* y necesita que las tres VMs estén ya arrancadas para escribir las tres entradas en `~/.ssh/config` del *host*. Sin `NO_PARALLEL`, Vagrant podría lanzar el *trigger* antes de que `vagrant ssh-config` devolviera info de los tres nodos.

Provisioners run: "never". Las fases pesadas (`setup-kubespray`, `setup-lab`, `setup-security`) están marcadas como `run: "never"`: no se ejecutan en `vagrant up`. Se invocan manualmente:

```

1 vagrant provision test-master --provision-with setup-kubespray
2 vagrant provision test-master --provision-with setup-lab
3 vagrant provision test-master --provision-with setup-security

```

Esto permite reconstruir la infraestructura (`vagrant destroy -f && vagrant up`) en pocos minutos sin pagar el coste de re-instalar Kubespray (10 min) o re-desplegar el *stack* de monitoring. Cada *provisioner* es idempotente, así que puede re-ejecutarse sin romper estado.

1.6 Provisioners de fase 0 (config.sh y ssh-setup.sh)

Dos *provisioners shell* corren automáticamente en cada `up` dentro de las tres VMs.

1.6.1 config.sh

Instala los paquetes base que necesitan las fases posteriores (`python3` y `python3-venv` para Kubespray, `git` para clonar repositorios, `vim` para edición manual). Es deliberadamente minimalista: Kubespray y Helm tienen sus propios bootstrap.

1.6.2 ssh-setup.sh

Crea un usuario personalizado por VM (`master`, `worker1`, `worker2`) con `sudo NOPASSWD`, copia la clave pública de Vagrant a su `authorized_keys` (para que el SSH desde el *host* funcione con la misma clave de Vagrant) e instala una *clave compartida del lab* en todas las VMs:

- La clave privada `Ed25519` se incrusta como *heredoc* en el *script*. Se autoriza también en `authorized_keys` de cada usuario.
- Resultado: `master@test-master` puede hacer `ssh worker1@10.0.1.11` sin contraseña ni copia de claves manual.

Este mecanismo es el que después usan tanto Kubespray (Ansible necesita SSH *passwordless* hacia los *workers*) como `setup-lab.sh` (que entra por SSH a `worker2` para hacer `chown` de los `hostPath`). El que la clave esté *hardcodeada* en el *script* es aceptable porque el lab es local y la red no está expuesta.

1.7 Trigger ssh-config.sh

Se ejecuta en el *host* tras levantar *test-worker2* (la última VM). Genera entradas SSH del estilo:

```
1 Host test-master
2   HostName 10.0.1.10
3   User master
4   Port 22
5   IdentityFile /home/USER/.vagrant.d/insecure_private_key
```

dentro de ~/.ssh/config, envueltas entre los marcadores # BEGIN paralelo-proyecto / # END paralelo-proyecto. La idempotencia se garantiza con *awk*: en cada ejecución borra el bloque anterior y lo reescribe. El usuario que aparece en *User* es el personalizado (*master/worker1/worker2*), no *vagrant*, mediante un *awk* que sustituye *User vagrant* por el correcto según el *Host*.

A partir de este punto el operador puede hacer *ssh test-master* desde el *host* y entrar directamente como *master* con la clave de *Vagrant*.

1.8 Discos persistentes

1.8.1 Decisión arquitectónica

Los discos *qcow2* **viven fuera de vagrant-libvirt**: son creados, formateados y montados por *scripts* propios y no aparecen en el *Vagrantfile*. La razón es de durabilidad: *vagrant destroy -f* borra todo el almacenamiento que gestiona *vagrant-libvirt* (el disco principal de cada VM, los *box* cacheados, etc.), pero los *qcow2* de *disks/* sobreviven porque no son suyos. Esto permite reconstruir el clúster manteniendo los datos de */mnt/data/grafana*, */mnt/data/prometheus*, */mnt/data/loki* entre destrucciones.

1.8.2 create-disks.sh (*trigger* en el host)

Se ejecuta antes de levantar la primera VM, como *trigger.before :up only_on: test-master*. Crea tres archivos *qcow2 sparse* (la imagen pesa unos pocos MiB hasta que se escribe en el sistema de ficheros) dentro de *disks/*:

- *test-master-data.qcow2*: 40 GB
- *test-worker1-data.qcow2*: 20 GB
- *test-worker2-data.qcow2*: 20 GB

Antes de *qemu-img create*, el *script* configura ACLs (*setfacl -m u:libvirt-qemu:x \$HOME*) para que el usuario *libvirt-qemu* pueda atravesar el *\$HOME* del usuario del *host* y leer los *qcow2*. Esta ACL granular es preferible al *chmod o+x \$HOME*, que expondría todo el *home* al resto de usuarios del sistema.

1.8.3 attach-disks.sh (*trigger* en el host)

Se ejecuta tras levantar *test-worker2*. Es la fase con más lógica idempotente del proyecto y trabaja en dos planos: *host* (*virsh attach-disk*) y *VM* (mediante SSH *heredoc*). Estructura:

- **Resolución del ID de dominio libvirt**: lee *.vagrant/machines/<node>/libvirt/id* para obtener el UUID del dominio. Sin ese fichero, omite el nodo con un mensaje de error.
- **Comprobación de vdb ya conectado**: *virsh domblklist* muestra los discos del dominio; si *vdb* ya aparece, se omite la conexión.
- *virsh attach-disk ... --persistent*: la opción *--persistent* es clave. Sin ella, el *attach* se aplica solo al proceso *QEMU* vivo y se pierde en cualquier reinicio. Con *--persistent*, *libvirt* escribe el cambio al XML de definición del dominio y *vdb* reaparece tras un *virsh start* o un *reboot* del *host*.
- **Formateo dentro de la VM (SSH heredoc)**: si *blkid /dev/vdb* no encuentra sistema de ficheros, se formatea como *ext4*. Si ya existe, se respeta (idempotente).
- **Montaje persistente vía /etc/fstab**: se añade la línea */dev/vdb /mnt/data ext4 defaults,nofail 0 2*. La opción *nofail* es defensiva: si por cualquier razón el disco no aparece, la VM arranca igualmente en vez de quedarse en modo *emergency*.

- **Montaje en caliente:** `sudo mount /mnt/data` si no está ya montado.

El resultado es que cada nodo tiene un volumen ext4 estable bajo `/mnt/data`, listo para alojar `hostPath` de Kubernetes.

1.9 Acceso gráfico a las VMs

Las máquinas exponen consola gráfica VNC (driver `cirrus` por defecto en `generic/ubuntu2204`). Para acceder a una de ellas se puede usar:

```
1 virt-viewer --connect qemu:///system test-master
2 # o desde virt-manager: conexión QEMU/KVM > test-master > Open Console
```

1.10 Resumen del estado tras Fase 1

Tras un `vagrant up` exitoso, el estado del laboratorio es:

- Red `red-prueba` (`virbr2`) operativa con NAT y reserva DHCP por MAC.
- Tres VMs Ubuntu 22.04 con sus IPs estáticas (10.0.1.10/11/12).
- Usuarios personalizados `master`, `worker1`, `worker2` con `sudo NOPASSWD` y clave compartida del lab para SSH entre nodos.
- Acceso SSH desde el *host* mediante alias (`ssh test-master/worker1/worker2`).
- Disco persistente `qcow2` montado en `/mnt/data` en cada VM, listo para `hostPath` en fases posteriores.

A partir de aquí, los *provisioners* `run: never` se invocan en el orden `setup-kubespray` → `setup-lab` → `setup-security`.

2 Fase 2 – Despliegue del clúster Kubernetes

2.1 Objetivo

Sobre las tres VMs aprovisionadas en la Fase 1 se despliega un clúster Kubernetes de tres nodos (un *control-plane* y dos *workers*) usando **Kubespray** v2.26.0. El resultado es un clúster Kubernetes v1.30.4 con CNI Flannel, container runtime `containerd` y `etcd stacked` (corriendo en el mismo *control-plane*). Toda la fase se ejecuta desde `test-master` de forma idempotente mediante el *script* `scripts/setup-kubespray.sh`.

2.2 Por qué Kubespray

Las alternativas evaluadas fueron:

- **kubeadm** (oficial, manual): requiere *bootstrap* explícito por nodo. Para tres nodos es razonable, pero la curva de mantenimiento es alta cuando hay que cambiar versión o re-aplicar configuración.
- **k3s**: distribución *lightweight* con un único binario. Excelente para edge, pero esconde decisiones (`etcd` reemplazado por SQLite por defecto, CNI customizado) que en un proyecto formativo conviene mantener visibles.
- **Kubespray** (elegida): *collection* de Ansible mantenida por SIGs de Kubernetes. Despliega Kubernetes *vanilla upstream* con componentes intercambiables vía variables. La curva de aprendizaje de Ansible se rentabiliza por la trazabilidad (cada cambio de versión es un `git checkout` y un *re-run*).

2.3 Estructura del *script* setup-kubespray.sh

El *script* se ejecuta como *provisioner shell* en *test-master* con `privileged: false` (es decir, como usuario *vagrant*) y *re-execs* a sí mismo como el usuario personalizado *master* mediante:

```
1 if [ "$(id -un)" != "master" ]; then
2   chmod a+r "$0"
3   exec sudo -iu master bash "$0" "$@"
4 fi
```

Esto resulta en que todo el estado (`~/kubespray`, `~/kubespray-venv`, `~/.kube/config`) queda en `/home/master/`, no en `/home/vagrant/`. El `chmod a+r` previo es necesario porque Vagrant sube el *script* a `/tmp` con modo 700 de propietario *vagrant*; sin ese permiso, *master* no podría siquiera leerlo.

El *script* se organiza en seis fases idempotentes.

2.3.1 Fase 1: Clonado de Kubespray

```
1 KUBESPRAY_TAG="v2.26.0"
2 git clone --branch "$KUBESPRAY_TAG" --depth 1 \
3   https://github.com/kubernetes-sigs/kubespray.git $HOME/kubespray
```

Se fija una versión concreta (v2.26.0, que despliega Kubernetes 1.30.4) en lugar de *master*. Si en una segunda ejecución el directorio ya existe, el *script* hace `fetch --tags + checkout` sobre el tag fijado — garantiza que la versión usada es siempre la misma.

2.3.2 Fase 2: Entorno virtual de Python

Kubespray requiere Ansible y un conjunto de dependencias Python específicas. Para no contaminar el Python del sistema:

```
1 python3 -m venv $HOME/kubespray-venv
2 source $HOME/kubespray-venv/bin/activate
3 pip install -q -U pip
4 pip install -q -r $HOME/kubespray/requirements.txt
```

2.3.3 Fase 3: Generación del inventario

El inventario `hosts.yml` se escribe *inline* mediante *heredoc*, no copiando `inventory/sample`. La razón es que cada host declara su `ansible_user` explícito (*master*, *worker1*, *worker2*) en vez de un usuario global vía `--user`. Esto evita conflictos cuando Ansible necesita conectar con diferentes usuarios según el nodo.

```
1 all:
2   hosts:
3     master:
4       ansible_host: 10.0.1.10
5       ansible_user: master
6       ip: 10.0.1.10
7       access_ip: 10.0.1.10
8     worker1:
9       ansible_host: 10.0.1.11
10      ansible_user: worker1
11      ip: 10.0.1.11
12      access_ip: 10.0.1.11
13     worker2:
14       ansible_host: 10.0.1.12
15       ansible_user: worker2
16       ip: 10.0.1.12
17       access_ip: 10.0.1.12
18   children:
19     kube_control_plane:
20       hosts: { master: }
21     kube_node:
22       hosts: { worker1:, worker2: }
23     etcd:
24       hosts: { master: }
25     k8s_cluster:
26       children:
```

```

27     kube_control_plane:
28     kube_node:
29     calico_rr:
30     hosts: {}

```

Los grupos `kube_control_plane`, `kube_node`, `etcd` son nombres que Kubespray usa internamente para decidir qué roles aplicar a qué hosts. `calico_rr` se declara vacío porque no usamos Calico Route Reflectors.

2.3.4 Fase 4: Configuración de `group_vars`

```

1 cp -r $HOME/kubespray/inventory/sample/group_vars/. \
2     $HOME/kubespray/inventory/cluster-k8-paralelo/group_vars/
3 sed -i 's/^kube_network_plugin:.*k8s_cluster/kube_network_plugin: flannel/' \
4     $HOME/kubespray/inventory/cluster-k8-paralelo/group_vars/k8s_cluster/k8s-cluster.yml

```

Se copia el `group_vars` de muestra y se sustituye el CNI por **flannel**. El *default* de Kubespray es Calico, pero Flannel es más ligero y suficiente para un laboratorio: VXLAN sobre la red privada de libvirt, sin policias de red (que tampoco usaríamos en este proyecto).

2.3.5 Fase 5: Ejecución del *playbook*

```

1 cd $HOME/kubespray
2 ansible-playbook -i inventory/cluster-k8-paralelo/hosts.yml --become cluster.yml

```

El despliegue completo dura entre 8 y 12 minutos. Se ejecuta con `--become` para que Ansible escale a `root` en los *workers* cuando lo necesite (instalación de paquetes, configuración de `containerd`, etc.) — recordemos que los tres usuarios personalizados están configurados con `sudo NOPASSWD` en `ssh-setup.sh`.

2.3.6 Fase 6: Instalación de `kubeconfig`

```

1 mkdir -p $HOME/.kube
2 sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
3 sudo chown $(id -u):$(id -g) $HOME/.kube/config

```

Kubespray escribe el `kubeconfig` del clúster en `/etc/kubernetes/admin.conf` (propiedad de `root`). Lo copiamos al `~/.kube/config` del usuario `master` y le cambiamos el propietario para que `kubectl` funcione sin `sudo`.

2.3.7 Fase 7: Creación de *namespaces*

Antes de salir, el *script* crea los tres *namespaces* que usarán las fases posteriores:

```

1 kubectl create namespace vulnerable
2 kubectl create namespace monitoring
3 kubectl create namespace security

```

Aislamiento por dominio:

- **monitoring**: Grafana, Prometheus, Loki, Promtail.
- **security**: Falco, Suricata.
- **vulnerable**: DVWA, WebGoat, Juice Shop.

La separación `security` ↔ `vulnerable` es deliberada: si un compromiso de DVWA escala fuera del pod, no debe ver los *logs* de Falco y Suricata. En el lab no aplicamos `NetworkPolicies` (Flannel no las soporta sin un *plugin* adicional), pero la separación lógica ya facilita una transición futura a Calico.

2.4 Validación post-despliegue

Tras la ejecución, los comandos esperables:

```
1 $ kubectl get nodes
2 NAME          STATUS    ROLES          AGE    VERSION
3 master        Ready     control-plane   2m     v1.30.4
4 worker1       Ready     <none>          1m     v1.30.4
5 worker2       Ready     <none>          1m     v1.30.4
6
7 $ kubectl get pods -A
8 NAMESPACE     NAME                                READY   STATUS
9 kube-system    kube-flannel-...                   1/1     Running
10 kube-system    kube-proxy-...                     1/1     Running
11 kube-system    coredns-...                        1/1     Running
12 kube-system    nodelocaldns-...                   1/1     Running
```

2.5 Lecciones y *gotchas* encontradas

2.5.1 Re-exec a usuario master con `chmod a+r`

Vagrant sube el *script* a `/tmp/vagrant-shell` con modo 700 y propietario `vagrant`. Si el *script* hace `exec sudo -iu master bash "$0"` sin antes ampliar permisos, el *exec* falla porque `master` no puede leer el archivo. La línea `chmod a+r "$0"` resuelve el problema y es segura: el archivo está bajo `/tmp`, propiedad de `vagrant`, y dejar lectura abierta no expone secretos del repositorio.

2.5.2 Per-host `ansible_user`

Inicialmente la tentación es pasar `--user master` a `ansible-playbook` — es lo que sale en la mayoría de tutoriales. El problema es que el SSH desde `master` (donde corre Ansible) hacia los *workers* debe usar `worker1` y `worker2` como usuarios, no `master`. La solución elegante es declarar `ansible_user` *por host* en `hosts.yml` y no pasar `--user` globalmente.

2.5.3 Inter-node SSH *passwordless*

Kubespray funciona porque `ssh-setup.sh` ya ha pre-instalado la clave compartida del lab en `authorized_keys` de los tres usuarios. Si esa clave no estuviera en `master → worker1` y `master → worker2`, Ansible quedaría esperando interactivamente la contraseña — y el *provisioner* fallaría sin pista clara del motivo.

2.5.4 Idempotencia del clúster

Re-ejecutar `setup-kubespray.sh` sobre un clúster ya desplegado es seguro: `git fetch && checkout` es *no-op* si ya está en el tag; `python -m venv` respeta el directorio existente; `kubectl create namespace` fallaría con `AlreadyExists`, pero el patrón canónico es:

```
1 kubectl create namespace monitoring --dry-run=client -o yaml | kubectl apply -f -
```

Este patrón se usa en `setup-security.sh` (Fase 4) para asegurar *namespaces* de forma idempotente sin enmascarar errores genuinos con `|| true`.

3 Fase 3 – *Stack* de monitorización

3.1 Objetivo

Sobre el clúster desplegado en la Fase 2 se instala el *stack* de observabilidad clásico de la comunidad cloud-native: **Prometheus** para métricas, **Loki** para *logs*, **Grafana** como UI unificada, y **Promtail** como agente de *shipping* de *logs*. Todo se concentra físicamente en `test-worker2`, que aloja los `hostPath` de los tres servidores. La automatización completa vive en `scripts/setup-lab.sh`.

3.2 Arquitectura

Componente	Despliegue	Nodo	Acceso
Grafana	Deployment (1 <i>replica</i>)	worker2	NodePort 30711
Prometheus	Deployment (1 <i>replica</i>)	worker2	NodePort 30090
Loki	StatefulSet (SingleBinary)	worker2	ClusterIP 3100
Promtail	DaemonSet (sin master)	worker1, worker2	—
node-exporter	DaemonSet	los 3	ClusterIP 9100
kube-state-metrics	Deployment	cualquier worker	ClusterIP 8080

El flujo de datos es:

- **Métricas:** node-exporter + kube-state-metrics → Prometheus (*pull* cada 15 s) → Grafana (consultas PromQL).
- **Logs:** /var/log/pods/* → Promtail (*tail* en cada *worker*) → Loki (push HTTP) → Grafana (consultas LogQL).

3.3 Volúmenes persistentes vía hostPath

Los tres servidores (Grafana, Prometheus, Loki) usan `hostPath` en `/mnt/data/` de `test-worker2`. El disco persistente bajo `/mnt/data` es el `qcow2` montado en Fase 1, que sobrevive a `vagrant destroy`. Eso permite mantener *dashboards*, métricas e *logs* históricos entre reconstrucciones completas del clúster.

Los PV se definen estáticamente en `manifests/storage/`:

```
1 apiVersion: v1
2 kind: PersistentVolume
3 metadata:
4   name: grafana-pv
5 spec:
6   capacity: { storage: 1Gi }
7   accessModes: [ReadWriteOnce]
8   persistentVolumeReclaimPolicy: Retain
9   storageClassName: ""
10  hostPath:
11    path: /mnt/data/grafana
12 ---
13 apiVersion: v1
14 kind: PersistentVolumeClaim
15 metadata:
16   name: grafana-pvc
17   namespace: monitoring
18 spec:
19   accessModes: [ReadWriteOnce]
20   storageClassName: ""
21   resources: { requests: { storage: 1Gi } }
22   volumeName: grafana-pv
```

El `volumeName: grafana-pv` en el PVC fuerza el *binding* **estático** con el PV concreto (en vez del *binding* dinámico por `StorageClass`, que aquí no aplica porque ambos tienen `storageClassName: ""`).

Caso especial Loki. El PV de Loki se declara *sin* su PVC: Loki corre como `StatefulSet` y el *chart* genera el PVC automáticamente vía `volumeClaimTemplates` con el nombre `storage-loki-0`. El *binding* ocurre por *matching* de `capacity`, `accessModes` y `storageClassName`, no por `volumeName` explícito.

Permisos chown. Antes de cualquier `helm install`, el *script* entra por SSH a `worker2` y hace:

```
1 sudo mkdir -p /mnt/data/grafana /mnt/data/prometheus /mnt/data/loki
2 sudo chown 472:472 /mnt/data/grafana # UID 'grafana' en la imagen
3 sudo chown 65534:65534 /mnt/data/prometheus # UID 'nobody'
4 sudo chown 10001:10001 /mnt/data/loki # UID del podSecurityContext del chart
```

Esto es **obligatorio**. `fsGroup` en el `podSecurityContext` *no recursiza sobre hostPath*, así que el `chown` a posteriori (o por un *init container* `busybox`) es la única manera de que el proceso del contenedor (no privilegiado) pueda escribir. En los tres *charts* desactivamos el `initChownData` para no depender del *pull* de `docker.io/library/busybox` desde el registro público.

3.4 Grafana

3.4.1 Despliegue

Se instala con el *chart* oficial grafana/grafana. Los valores relevantes:

```
1 adminUser: admin
2 adminPassword: admin
3 persistence:
4   enabled: true
5   existingClaim: grafana-pvc
6 service:
7   type: NodePort
8   nodePort: 30711
9 nodeSelector:
10   kubernetes.io/hostname: worker2
11 initChownData:
12   enabled: false
```

3.4.2 Provisioning declarativo de *datasources*

En lugar de cablear los *datasources* contra la API de Grafana *a posteriori* (POST /api/datasources), se declaran dentro del propio *values.yaml*:

```
1 datasources:
2   datasources.yaml:
3     apiVersion: 1
4     deleteDatasources:
5       - { name: Prometheus, orgId: 1 }
6       - { name: Loki, orgId: 1 }
7     datasources:
8       - name: Prometheus
9         uid: prometheus
10        type: prometheus
11        access: proxy
12        url: http://prometheus-server.monitoring.svc.cluster.local
13        isDefault: true
14       - name: Loki
15         uid: loki
16        type: loki
17        access: proxy
18        url: http://loki.monitoring.svc.cluster.local:3100
```

El *chart* genera un *ConfigMap* con este archivo y lo monta en */etc/grafana/provisioning/datasources/*. Grafana lo lee al arrancar el pod y reconcilia. Ventajas: versionado en *git* (*single source of truth*), idempotente, los *datasources* aparecen marcados como *Provisioned* en la UI con candado de solo lectura.

El bloque deleteDatasources (defensivo). Una de las lecciones más importantes del proyecto. El *flow* de fallo es:

1. Primer *install* sin *uid*: explícito → Grafana asigna UUIDs aleatorios y los persiste en la *sqlite* del PVC.
2. Se introduce *uid: prometheus* / *uid: loki* en una segunda iteración.
3. Al rearrancar el pod, el *provisioner* busca por *uid* → no encuentra (la entrada vieja tiene UUID diferente) → intenta *insert* → colisiona por *unique constraint* en (*name*) → *CrashLoopBackOff* con *logs* crípticos: “*data source not found*”.

deleteDatasources se ejecuta *antes* del bloque *datasources* en cada arranque, borrando por (*name*, *orgId*). En arranques posteriores el *delete* es *no-op* (la entrada ya está alineada al *uid* declarado) y el *insert* reaplica los mismos valores. Coste: un par de líneas en *logs* por arranque. Beneficio: el pod nunca queda atrapado en *CrashLoopBackOff* por una *sqlite* heredada con UUIDs inconsistentes.

3.4.3 Provisioning declarativo de *dashboards*

Mismo patrón aplicado a *dashboards*: se declaran en el `values.yaml` y el *chart* genera un ConfigMap por *dashboard*. El *sidecar* `kiwigrid/k8s-sidecar` inyectado por el *chart* monta el ConfigMap en `/var/lib/grafana/dashboards` y Grafana los carga al arrancar.

```
1 dashboardProviders:
2   dashboardproviders.yaml:
3     apiVersion: 1
4     providers:
5       - name: 'lab'
6         orgId: 1
7         folder: 'Lab'
8         type: file
9         disableDeletion: false
10        editable: true
11        options:
12          path: /var/lib/grafana/dashboards/lab
13
14 dashboards:
15   lab:
16     security-overview:
17       json: |
18       { ... JSON inline del dashboard ... }
```

El *dashboard* `security-overview` (15 paneles) reúne en una sola vista las métricas de Prometheus (CPU/memoria/red/disco por nodo, filesystem %) y los *logs* de Loki (eventos Falco y Suricata, top firmas, tráfico por CMS), todos referenciados por `uid: prometheus` / `uid: loki` para que el *provisioning* reproduzca el *dashboard* con *datasources* estables.

Paneles “% por nodo”. Las queries de los paneles *multi-nodo* usan `avg by (node) (...)` en vez de `by (node)` a secas. Es una defensa preventiva: si un *rollout* o un cambio de *scrape* deja series duplicadas temporalmente (mismo nodo, distintos labels de *job*), `avg by` las colapsa en una sola línea. Sin sesgo cuando los valores son idénticos, evita el artefacto visual de ver “*master/master/worker1/worker1/...*” en la leyenda durante los 5 minutos de *staleness* de la TSDB.

3.5 Prometheus

Se instala con el *chart* `prometheus-community/prometheus` en *scope medium*: solo servidor + node-exporter + kube-state-metrics. Alertmanager y Pushgateway desactivados.

```
1 server:
2   persistentVolume:
3     enabled: true
4     existingClaim: prometheus-pvc
5   initChownData:
6     enabled: false
7   service:
8     type: NodePort
9     nodePort: 30090
10    nodeSelector:
11      kubernetes.io/hostname: worker2
12    retention: "7d"
13  alertmanager:
14    enabled: false
15  prometheus-pushgateway:
16    enabled: false
```

El bug del *double scrape*. Una de las gotchas más sutiles del proyecto. El *chart* de Prometheus incluye por defecto el *job* `kubernetes-service-endpoints`, que auto-descubre cualquier *Service* con la anotación `prometheus.io/scrape=true`. El *Service* del *node-exporter* ya trae esa anotación, así que se *scrapes* automáticamente.

Si además se declara un *extraScrapeConfigs* personalizado para *relabel* (por ejemplo, exponer `__meta_kubernetes_pod_node_name` como *node*), el mismo endpoint `:9100` se *scrapes* *dos veces* — una por cada *job* — y la TSDB acaba con dos series con labels distintos pero valores casi idénticos. El síntoma en Grafana son leyendas duplicadas (*master/master/worker1/worker1/...*) durante los ~5 min de retención de *staleness* de Prometheus.

Solución adoptada. Eliminar `extraScrapeConfigs` y dejar solo el *auto-discovery*, que ya hace el *relabel* de `node` en versiones recientes del *chart*. El bloque se conserva como comentario en el `values.yaml` con valor pedagógico:

```
1 # extraScrapeConfigs: |
2 #   - job_name: node-exporter-clean
3 #     kubernetes_sd_configs:
4 #       - role: endpoints
5 #         namespaces: { names: [monitoring] }
6 #     relabel_configs:
7 #       - source_labels: [__meta_kubernetes_service_name]
8 #         regex: prometheus-prometheus-node-exporter
9 #         action: keep
10 #       - source_labels: [__meta_kubernetes_pod_node_name]
11 #         target_label: node
```

Gotcha `extraScrapeConfigs` es *top-level*. La estructura del *chart* pone esta clave a nivel *root*, no bajo `server:`. Helm la ignora silenciosamente si está mal anidada: el `ConfigMap` renderizado no la contiene, no hay error, y el operador descubre que falta solo al ver que el *job* nunca aparece en `/targets` de la UI de Prometheus.

3.6 Loki

Loki es el componente más sensible a tunear del *stack*. Se instala con el *chart* oficial `grafana/loki` en modo `SingleBinary`.

3.6.1 Modo SingleBinary

Es el *único* modo compatible con *filesystem storage*. Simple Scalable y Distributed requieren *object storage* (S3/GCS/MinIO) por restricción explícita del *chart*. Como en este *lab* solo tenemos `hostPath` en `worker2`, no es una decisión de simplicidad pedagógica: es una restricción técnica.

3.6.2 Configuración base

```
1 deploymentMode: SingleBinary
2
3 loki:
4   auth_enabled: false
5   commonConfig:
6     replication_factor: 1
7   storage:
8     type: filesystem
9   schemaConfig:
10    configs:
11      - from: "2024-04-01"
12        store: tsdb
13        object_store: filesystem
14        schema: v13
15        index: { prefix: loki_index_, period: 24h }
```

3.6.3 Sizing bajo pen-test flood

Una lección concreta del proyecto: con los *defaults* del *chart* (`limits.memory: 256Mi`), Loki entra en *OOMKill loop* durante un *pen-test* simulado (`nikto + gobuster + sqlmap` en paralelo contra los tres CMS, ~500 eventos/s). Los *limits* subidos a 768Mi cubren el escenario peor:

```
1 singleBinary:
2   resources:
3     requests: { cpu: 50m,  memory: 256Mi }
4     limits:   { cpu: 500m, memory: 768Mi }
```

La RAM se consume en:

- *ingest buffers* y escrituras al WAL.

- Decompresión de *chunks* durante las queries del *dashboard* (auto-refresh cada 5 s).
- Pattern matching con `label_format` / `hasPrefix` (paneles de status codes y top URIs).

3.6.4 Tamaño máximo de mensaje gRPC

```
1 loki:
2   server:
3     grpc_server_max_recv_msg_size: 33554432      # 32 MiB
4     grpc_server_max_send_msg_size: 33554432
5     http_server_read_timeout: 60s
6     http_server_write_timeout: 60s
```

El *default* es 4 MB. Bajo *flood*, las queries del *dashboard* (`count_over_time`, `topk`) devuelven *payloads* > 10 MB que rompen el bus gRPC interno con un error críptico (“*grpc: received message larger than max*”). 32 MiB cubre con margen.

3.6.5 Límites de *query*

```
1 loki:
2   limits_config:
3     retention_period: 168h          # 7 dias, coherente con Prometheus
4     split_queries_by_interval: 15m
5     max_query_series: 5000
6     max_entries_limit_per_query: 5000
```

`split_queries_by_interval` parte queries grandes en ventanas de 15 minutos que se ejecutan en paralelo internamente. Una query de 1 hora se vuelve 4 sub-queries más manejables, lo que evita picos de RAM transitorios al decomprimir un *chunk* entero.

3.7 Promtail

Promtail corre como `DaemonSet` en `worker1` + `worker2` (master excluido por `tolerations: []`). Lee de `/var/log/pods/*` en cada nodo y hace *push* HTTP a Loki.

```
1 config:
2   clients:
3     - url: http://loki.monitoring.svc.cluster.local:3100/loki/api/v1/push
4   tolerations: []
5   serviceMonitor:
6     enabled: false
```

El *override* crítico de la URL. El *default* del *chart* apunta a `http://loki-gateway/...` (el *nginx reverse proxy* del *chart* de Loki). Pero en nuestra instalación `gateway.enabled: false`, así que el `Service loki-gateway` no existe y Promtail entra en bucle de *retries*. Apuntando directo al `Service loki` se evita el salto intermedio.

`serviceMonitor: false`. El recurso `ServiceMonitor` pertenece al *Prometheus Operator* (CRD del *chart kube-prometheus-stack*). Aquí usamos Prometheus *standalone*, que no instala ese CRD. Si dejásemos `serviceMonitor.enabled: true` (el *default*), el *chart* intentaría crear el recurso y `helm` fallaría con “*no matches for kind ServiceMonitor*”.

Deprecation. El *chart grafana/promtail* está marcado como *deprecated* a favor de `grafana/alloy`. La decisión consciente fue mantener Promtail por ahora: el coste de migrar a Alloy en mitad del proyecto formativo es mayor que el beneficio (mismo *output* a Loki, distinta sintaxis de *pipeline*). Migración prevista para una segunda iteración del laboratorio.

3.8 Convención de *labels* en LogQL

Promtail con la configuración del *chart* genera *labels* cortos:

- namespace (no `kubernetes_namespace`).
- app (no `app.kubernetes.io/name` ni `kubernetes_pod_label_app`).
- pod, container, node.

Esto es deliberado: *labels* cortos son más legibles en queries LogQL y reducen cardinalidad de *stream*. Las queries canónicas del *dashboard* terminan siendo del estilo:

```
1 {namespace="security", app="falco"} | json | rule = "Terminal shell in container"
2 {namespace="security", app="suricata"} | json | event_type = "alert"
```

3.9 El *script* setup-lab.sh

Orquesta todo lo anterior en 8 fases idempotentes con `helm upgrade --install --wait`. Recupera la contraseña real del admin de Grafana desde el Secret (helm no rota Secrets en *upgrades*, así que el *values.yaml* solo aplica al primer *install*). Endpoints finales:

```
1 Grafana:      http://10.0.1.12:30711    (admin / admin)
2 Prometheus:  http://10.0.1.12:30090
3 Loki:        solo interno (http://loki.monitoring.svc.cluster.local:3100)
```

Invocación:

```
1 vagrant provision test-master --provision-with setup-lab
```

4 Fase 4 – *Stack* de seguridad y CMS vulnerables

4.1 Objetivo

Sobre el clúster con monitorización ya activa se despliegan dos detectores y tres aplicaciones vulnerables. Los detectores son **Falco** (HIDS basado en eBPF, observa *syscalls* dentro de cualquier *container*) y **Suricata** (NIDS basado en *af_packet*, observa tráfico de red en *eth0* de un nodo). Las aplicaciones vulnerables son **DVWA**, **WebGoat** y **Juice Shop** — tres CMS clásicos del catálogo OWASP que servirán como blancos de *pen-testing* simulado desde una Kali externa al clúster.

Toda la fase se automatiza desde `scripts/setup-security.sh`, que comparte el patrón de `setup-lab.sh`: *re-exec* como *master*, `helm upgrade --install --wait`, idempotencia por diseño.

4.2 Arquitectura

Componente	Despliegue	Nodo	Acceso
Falco	DaemonSet (sin master)	worker1, worker2	—
Suricata	DaemonSet (nodeSelector)	worker1	hostNetwork eth0
DVWA	Deployment 1 <i>replica</i>	worker1	NodePort 30430
WebGoat	Deployment 1 <i>replica</i>	worker1	NodePort 30380
Juice Shop	Deployment 1 <i>replica</i>	worker1	NodePort 31592

Pipeline de eventos:

- **Falco**: `stdout JSON` → Promtail (ya desplegado) → Loki → Grafana.
- **Suricata**: `eve.json` (archivo) → `sidecar tail -F` → `stdout` → Promtail → Loki → Grafana.
- **Tráfico atacante**: Kali (10.0.1.20, externa al cluster CIDR) → *eth0* worker1 → visto por Suricata (*hostNetwork*) y por los *logs* de Apache/Spring/Node de cada CMS.

4.3 Concentración en worker1

Los tres CMS y Suricata corren en el mismo nodo. Razón técnica: Suricata captura tráfico con `hostNetwork: true` desde `eth0` del nodo donde corre. Si DVWA migrara a otro nodo (por re-scheduling tras una caída), Suricata dejaría de ver el tráfico atacante. Concentrar todo en worker1 es la garantía simple de que las firmas siempre disparen.

Trade-off: worker1 (3072 MiB de RAM) opera al 80–90% de uso con todo el *stack* activo. Los `resources.limits` de cada componente están ajustados para minimizar *OOMKill* bajo carga:

- Falco: 512 Mi limit (default 1 Gi).
- Suricata: 1 Gi limit (default 256 Mi — insuficiente con ET Open completo cargado).
- DVWA: 256 Mi limit (MySQL embebido es el componente más pesado).
- WebGoat: 512 Mi limit (Spring Boot + heap acotado por `-Xmx384m`).
- Juice Shop: 192 Mi limit (Node + Express + sqlite efímera, austero).

4.4 Falco

Falco se instala con el *chart* oficial `falcosecurity/falco` (8.0.2, app 0.43.1).

4.4.1 Driver modern_ebpf

```
1 driver:
2   kind: modern_ebpf
```

`modern_ebpf` usa `BPF_PROG_TYPE_RAW_TRACEPOINT`, lo que requiere kernel `>= 5.8`. Ubuntu 22.04 cumple (kernel 5.15). Ventajas frente al *kernel module* tradicional o al *eBPF legacy probe*:

- Sin `kernel-headers` en los *workers*: no hay compilación local de un módulo.
- Sin dependencia de `dkms`: el *upgrade* del kernel no rompe la instalación de Falco.
- *Self-contained*: el binario eBPF se compila *ahead-of-time* y se carga vía `syscall bpf()`.

4.4.2 Salida JSON a stdout

```
1 falco:
2   json_output: true
3   json_include_output_property: true
4   json_include_tags_property: true
5   log_stderr: true
6   log_syslog: false
7   stdout_output: { enabled: true }
8   file_output:   { enabled: false }
9   http_output:   { enabled: false }
```

Solo `stdout`, que kubelet escribe en `/var/log/pods/.../falco/0.log` → Promtail → Loki. `file_output` saturaría `hostPath` sin aportar nada (los eventos ya están en Loki); `http_output` requeriría un *receiver* externo (*falcosidekick*, que dejamos desactivado por la misma razón).

4.4.3 El parámetro `base_syscalls.repair`

```
1 falco:
2   base_syscalls:
3     repair: true
```

Sin esta línea, los *rules built-in* que filtran por `proc.tty != 0` (como `Terminal shell in container`) **nunca disparan**, porque `proc.tty` se queda siempre en 0 — el árbol de procesos no tiene los datos para poblarlo. Es un *footgun* silencioso: el *ruleset* carga, los eventos se ven en `kubectl logs`, pero las reglas *built-in* críticas no disparan nunca.

Con `repair: true` Falco analiza el *ruleset*, infiere qué *syscalls* auxiliares necesita (`clone`, `fork`, `setsid`, `procexit`, `ioctl`...) para poblar `proc.tty`, `proc.pname`, `proc.cmdline`, `container.id`, etc., y se suscribe automáticamente.

4.4.4 Regla custom: “Lab — Shell en container”

```
1 customRules:
2   rules-lab.yaml: |-
3     - rule: Lab -- Shell en container
4       desc: >
5         Dispara cuando un shell (bash, sh, zsh, ...) se ejecuta dentro
6         de un container. Excluye casos legitimos donde un package manager
7         o el runtime spawna shells como hooks de instalacion.
8       condition: >
9         spawned_process and container
10        and proc.name in (bash, sh, zsh, ksh, csh, dash, ash, fish)
11        and not proc.pname in (apt, apt-get, dpkg, dpkg-reconfigure, yum,
12                               dnf, apk, pip, npm, conda, sudo)
13       output: >
14         [LAB] Shell en container
15         (pod=%k8s.pod.name ns=%k8s.ns.name container=%container.name
16          shell=%proc.name cmdline=%proc.cmdline parent=%proc.pname tty=%proc.tty)
17       priority: WARNING
18       tags: [lab, shell]
```

Iteración hasta la regla final. La versión inicial (*canary* de validación E2E del *probe*) era `spawned_process` and `container`, sin filtros adicionales. Funcionaba como prueba de vida: cualquier `execve` dentro de cualquier *container* disparaba un evento. El problema observado:

- Flannel ejecuta `iptables` y `ip` continuamente.
- `kube-proxy` sondea reglas y servicios cada pocos segundos.
- Los *hooks* de `apt/dpkg` durante actualizaciones automáticas lanzan *shells* legítimos.

Resultado: ~16 eventos por segundo en estado *idle*, casi todos ruido del *runtime*. En 10 minutos, ~9656 eventos — imposible distinguir un atacante real.

Refinamiento. Se filtró por `proc.name in (bash, sh, zsh, ...)` para quedarse con *shells* interactivas, y se excluyó `proc.pname in (apt, dpkg, sudo, ...)` para descartar *shells* legítimas como hijos de gestores de paquetes. La regla refinada genera 0 eventos en estado *idle* y dispara puntualmente cuando un humano hace `kubectl exec - sh`.

4.4.5 customRules y rules_files: la trampa

```
1 falco:
2   rules_files:
3     - /etc/falco/falco_rules.yaml
4     - /etc/falco/falco_rules.local.yaml
5     - /etc/falco/rules.d
```

El *chart* monta `customRules` como `ConfigMap` en `/etc/falco/rules.d/`. **Crítico:** `/etc/falco/rules.d` debe estar en `rules_files`; si se sobrescribe `rules_files` sin incluirla, las reglas *custom* se montan en el pod pero Falco *nunca las carga*, sin error visible. Es la misma categoría de fallo que el `base_syscalls.repair`: el síntoma es “mi regla no dispara” y la causa está enterrada en una sub-clave de configuración.

4.5 Suricata

Suricata no tiene *chart* oficial estable. Se desarrolló un *chart* propio en `charts/suricata/` con tres recursos: `ConfigMap` (`suricata.yaml` + `update.yaml`), `DaemonSet` (`init` + `main` + `sidecar`) y los *helpers* de Helm.

4.5.1 hostNetwork y capabilities

```
1 hostNetwork: true
2 dnsPolicy: ClusterFirstWithHostNet
3 securityContext:
4   capabilities:
5     add: [NET_ADMIN, NET_RAW, SYS_NICE]
```

`hostNetwork: true` es el *toggle* crítico: el pod comparte el *network namespace* del nodo y ve `eth0` entera. Sin esto, Suricata solo ve el *netns* vacío del pod y captura 0 paquetes útiles.

Las *capabilities* mínimas son:

- `NET_ADMIN`: setear modo promiscuo, configurar el *ring buffer* `TPACKET_V3`.
- `NET_RAW`: crear *sockets* `AF_PACKET` (capa 2, Ethernet completo).
- `SYS_NICE`: cambiar prioridad de *threads* de captura.

Notablemente, **no se usa** `privileged: true`. El kernel Linux permite todo lo necesario con *capabilities* granulares, sin dar acceso total al host.

`dnsPolicy: ClusterFirstWithHostNet`. Cuando `hostNetwork: true`, el pod hereda `/etc/resolv.conf` del nodo (que no apunta a CoreDNS). Si tuviéramos que resolver *services* de Kubernetes desde dentro del pod, fallarían. Con `ClusterFirstWithHostNet` se fuerza el uso de CoreDNS aunque la red sea del host. En el lab Suricata no resuelve nombres K8s, pero la línea queda como higiene.

4.5.2 Captura con af-packet

```
1 af-packet:
2   - interface: eth0
3     cluster-id: 99
4     cluster-type: cluster_flow
5     defrag: yes
6     use-mmap: yes          # ring buffer en memoria compartida
7     tpacket-v3: yes        # API moderna del kernel
```

Modo IDS pasivo (no IPS). Para IPS habría que usar `nfqueue` y enganchar el tráfico vía `iptables -j NFQUEUE`, lo que no funciona bien en *containers* y queda fuera de *scope* del lab.

4.5.3 Variables de red: HOME_NET y HTTP_PORTS

```
1 vars:
2   address-groups:
3     HOME_NET: "[10.0.1.10/32,10.0.1.11/32,10.0.1.12/32,10.233.0.0/16]"
4     EXTERNAL_NET: "!"$HOME_NET"
5   port-groups:
6     HTTP_PORTS: "80,30430,30380,31592"
```

Diseño del HOME_NET. Las firmas ET Open están escritas en términos de `$EXTERNAL_NET -> $HOME_NET`: solo disparan cuando origen y destino tienen ese sentido. Si la IP de la Kali atacante (10.0.1.20) cayese dentro de `HOME_NET`, las firmas *no dispararían* (sería `HOME -> HOME`). Por eso se enumeran nodos individuales (/32) y el CIDR de pods/services (10.233.0.0/16) explícitamente, dejando 10.0.1.20 fuera. Era el *footgun* clásico: usar 10.0.1.0/24 “por simplicidad” anula la mayoría de detecciones sin error visible.

HTTP_PORTS parametrizado. Las firmas ET `WEB_SERVER ...` están escritas como `flow:to_server,established;http.uri; ...`, lo que requiere que Suricata haya clasificado el flujo como HTTP *antes* de evaluar la regla. La clasificación se hace por `$HTTP_PORTS`. Si un CMS expone en un puerto que no está en esa lista, Suricata lo trata como TCP genérico y las firmas web nunca disparan — de nuevo, sin error visible.

Cuando se añadieron WebGoat (30380) y Juice Shop (31592), tuvieron que añadirse a `HTTP_PORTS` para que las firmas web disparasen contra ellos. Para no repetir el error, la lista vive como variable en `values.yaml` del *chart*:

```

1 suricata:
2   httpPorts:
3     - 80
4     - 30430
5     - 30380
6     - 31592

```

y el ConfigMap la materializa con la función join de Sprig:

```

1 HTTP_PORTS: "{{ join \"\", \"\" .Values.suricata.httpPorts }}"

```

4.5.4 El *init container*: descarga del *ruleset* ET Open

```

1 initContainers:
2   - name: rules-update
3     image: jasonish/suricata:latest
4     command: ["suricata-update"]
5     args:
6       - "--no-test"
7       - "--output=/var/lib/suricata/rules"
8       - "--data-dir=/var/lib/suricata"
9       - "--config=/etc/suricata/update.yaml"

```

`suricata-update` viene en la imagen oficial. El *ruleset* se descarga a un `emptyDir` compartido con el *container* principal (no se persiste — se re-descarga en cada *start* del pod, ~30s).

URL directa frente al alias `et/open`.

```

1 update.yaml: |
2   sources:
3     - https://rules.emergingthreats.net/open/suricata-8.0.0/emerging.rules.tar.gz

```

El alias `et/open` requiere que `suricata-update update-sources` haya cargado el catálogo de fuentes *antes* del *fetch*. Ese catálogo se guarda en `--data-dir`, que en nuestro setup es un `emptyDir` nuevo cada arranque. Como el *init container* no ejecuta `update-sources`, el alias no resuelve — se intenta interpretar como URL literal y falla con `unknown url type: 'et/open'`. Pero el *init container* sale con `exit 0` (*warning*, no error), así que el pod queda `Ready` con solo 436 reglas *decoder/protocol* y **cero firmas ET Open**. Detección silenciosamente rota. Fijar la URL directa elimina el problema.

4.5.5 El *sidecar* `tail -F eve.json`

```

1 - name: log-tail
2   image: jasonish/suricata:latest
3   command: ["sh", "-c"]
4   args:
5     - |
6       until [ -f /var/log/suricata/eve.json ]; do sleep 1; done
7       exec tail -F /var/log/suricata/eve.json
8   securityContext:
9     allowPrivilegeEscalation: false
10    capabilities: { drop: [ALL] }

```

Promtail solo lee `stdout` de los pods; Suricata escribe `eve.json` a archivo. El *sidecar* es el puente: `tail -F` (mayúscula) re-abre el archivo si Suricata lo rota (algo que `tail -f` minúscula *no* hace, se queda leyendo el *inode* antiguo eternamente).

Reuso de la imagen principal. El *sidecar* usa `jasonish/suricata` en vez de `busybox`. Razones: (1) `tail GNU` está en la imagen, (2) un solo *pull* y un solo conjunto de *layers* cacheados, (3) algunos *blobs* de `busybox` en Docker Hub se sirven desde Cloudflare R2 con IPs ocasionalmente no enrutables en la red *red-prueba*. Trade-off: el *sidecar* arranca con 139MB de imagen en vez de 5 MB, pero no añade RAM real (*layers* compartidos).

Defensa en profundidad. El *sidecar* solo lee — no necesita *capabilities* ni acceso de escritura. `drop: [ALL]` elimina cualquier privilegio remanente. Un compromiso del *sidecar* no puede manipular *logs* ni capturar tráfico.

4.5.6 Coherencia config: stats

```
1 outputs:
2   - eve-log:
3     enabled: yes
4     types:
5       - alert
6       - http
7       - dns
8       - tls
9       - flow
10    # NO incluir 'stats' aqui si outputs.stats.enabled = no
11   - stats:
12     enabled: no
```

Una particularidad de Suricata: si `outputs.stats.enabled: no` globalmente, no se puede incluir `stats` dentro de `eve-log.types`. Suricata aborta el arranque con “*stats are disabled globally*”. La coherencia entre el *sub-output* y el *output* principal es obligatoria.

4.6 DVWA — *Damn Vulnerable Web Application*

DVWA se despliega vía *chart* propio en `charts/dvwa/`.

4.6.1 Imagen vulnerables/web-dvwa

```
1 image:
2   repository: vulnerables/web-dvwa
3   tag: latest
```

Se eligió la imagen `vulnerables/web-dvwa` (todo-en-uno: Apache + PHP + MySQL embebido) en lugar de `ghcr.io/digininja/dvwa` (la oficial, que requiere MariaDB como segundo `Deployment`). Razón: un solo *container*, menos partes móviles, BD inicializada *out of the box*.

4.6.2 Sin persistencia (decisión consciente)

```
1 persistence:
2   enabled: false
```

La imagen `vulnerables/web-dvwa` trae la base de datos MariaDB **horneada en la imagen** (en `/var/lib/mysql`). Su *entrypoint* (`/main.sh`) solo hace `service mysql start`, no ejecuta `mysql_install_db`. Si montamos un PVC encima de `/var/lib/mysql`, el *mount* “ensombrece” la BD pre-cocinada → *datadir* vacío → `mysqld` arranca pero falla con “*Table 'mysql.user' doesn't exist*”.

Alternativas descartadas: pre-popular el PV con un *init container* que copie la BD baked, o ejecutar `mysql_install_db` manualmente. Ambas añaden complejidad sin beneficio práctico — en un lab de detección de ataques, perder progreso entre *restarts* es aceptable.

4.6.3 Probe /login.php

```
1 probes:
2   liveness:
3     path: /login.php
4     initialDelaySeconds: 30
5     periodSeconds: 30
```

El *probe* apunta a `/login.php`, no a `/`. Razón: `/` devuelve 302 redirigiendo a `/setup.php` hasta que la BD se inicializa. `/login.php` devuelve 200 directamente con la BD horneada operativa al arranque.

4.6.4 strategy: Recreate

```
1 strategy:
2   type: Recreate
```

Aunque no hay PVC, se mantiene el patrón por consistencia con cualquier futuro *enable* de *persistence*. Con `Recreate`, el pod viejo se mata antes de arrancar el nuevo — no hay dos `mysqld` escribiendo a la misma BD simultáneamente.

4.7 WebGoat

4.7.1 Imagen oficial

```
1 image:
2   repository: webgoat/webgoat
3   tag: latest
```

webgoat/webgoat es la imagen oficial OWASP. Lleva WebGoat (puerto 8080) y WebWolf (puerto 9090) en un único *container*, lanzados por el *entrypoint* del JAR.

4.7.2 Variables de entorno críticas

```
1 env:
2   - { name: WEBGOAT_HOST, value: "0.0.0.0" }
3   - { name: WEBGOAT_PORT, value: "8080" }
4   - { name: WEBWOLF_HOST, value: "0.0.0.0" }
5   - { name: WEBWOLF_PORT, value: "9090" }
6   - { name: JAVA_TOOL_OPTIONS, value: "-Xmx384m -Xms256m" }
```

WEBGOAT_HOST=0.0.0.0. Sin esto, el JAR bindea solo a 127.0.0.1 y el NodePort responde con “*Connection refused*”: el tráfico llega al *container* pero no a ningún socket abierto. Es un *default* desafortunado de la imagen oficial.

JAVA_TOOL_OPTIONS para acotar el *heap*. Sin acotar, Spring Boot crece hasta donde le deja el host — en worker1, eso significa rebasar el limit de 512 Mi del *container* y morir OOMKilled (*exit 137*) tras pocos minutos. Acotar -Xmx384m reserva ~128 Mi para *metaspaces*, *thread stacks* y código JIT. Cualquier JVM moderna lee JAVA_TOOL_OPTIONS del entorno automáticamente.

4.7.3 URL *case-sensitive*: /WebGoat

WebGoat usa /WebGoat con W y G mayúsculas en sus redirecciones internas. /webgoat devuelve 404. El *probe* canónico:

```
1 probes:
2   liveness:
3     path: /WebGoat/login.mvc
4     initialDelaySeconds: 120
```

initialDelaySeconds: 120 cubre el *cold start* de Spring Boot (*classpath scanning* + inicialización de H2 embedded), que puede llegar a 90s en worker1 cargado.

4.8 Juice Shop

4.8.1 Despliegue mínimo

```
1 image:
2   repository: bkimminich/juice-shop
3   tag: latest
4 service:
5   type: NodePort
6   port: 3000
7   nodePort: 31592
8 persistence:
9   enabled: false
```

Juice Shop es el más austero del trío: Node + Express + sqlite efímera. No requiere variables de entorno específicas; la imagen oficial corre en modo *production* por defecto y los retos están todos accesibles desde /#/score-board.

4.8.2 Sin login inicial

A diferencia de DVWA (admin/password) y WebGoat (registro de cuenta nueva en cada arranque), Juice Shop expone toda la *landing page* sin autenticación. Los retos OWASP Top 10 están integrados como minijuegos progresivos.

4.9 El *script* setup-security.sh

Estructura en 9 fases:

1. **Precondición:** verifica que `monitoring/promtail` esté `Ready`. Sin `Promtail`, los eventos generados por `Falco` y `Suricata` se perderían — mejor abortar pronto que dejar detectores “huérfanos”.
2. **Repo Helm de Falco:** solo `Falco` viene de un repo *upstream*; `Suricata`, `DVWA`, `WebGoat` y `Juice Shop` son *charts* locales bajo `charts/`.
3. **Namespaces:** `security + vulnerable` con el patrón idempotente `create --dry-run=client -o yaml | apply -f -`.
4. **Falco.**
5. **Suricata:** `helm install /vagrant/charts/suricata`.
6. **DVWA:** `helm install /vagrant/charts/dvwa`.
7. **WebGoat:** `--timeout 8m` para cubrir el *cold start* de `Spring Boot`.
8. **Juice Shop.**
9. **Resumen:** imprime endpoints y queries `LogQL` canónicas.

Invocación:

```
1 vagrant provision test-master --provision-with setup-security
```

4.10 Lecciones transversales

4.10.1 Charts custom: límite a Helm *upstream*

Cuando no hay *chart* estable y bien mantenido (`Suricata`) o cuando el oficial impone más complejidad que beneficio (`DVWA` con la oficial `digininja/dvwa` + `MariaDB` aparte), escribir un *chart* propio simplificado de 3–4 plantillas es *más* robusto que parchear un *chart upstream* con `values`. El coste de mantenimiento es contenido: los *templates* son lineales y los `values` viven con el código que documentan.

4.10.2 Imágenes con *defaults* hostiles a `NodePort`

`WebGoat` es el ejemplo claro: una imagen “oficial” que bindea a `127.0.0.1` por defecto es prácticamente inservible en `Kubernetes` sin un `env` explícito de `0.0.0.0`. La lección: nunca asumir que una imagen oficial está optimizada para el caso *containerizado* — probar el *flow* `NodePort` → `pod` pronto y observar logs.

4.10.3 Detectores silenciosamente rotos

Tres ejemplos en este capítulo:

- `Falco` sin `base_syscalls.repair` → *rules built-in* con `proc.tty` nunca disparan.
- `Suricata` con alias `et/open` en `update.yaml` → *ruleset* `ET Open` no se descarga, queda con `436 decoder rules`.
- `Suricata` sin `HTTP_PORTS` actualizado → firmas `ET WEB_SERVER` no clasifican el flujo como `HTTP` y no disparan.

Los tres son síntomas de la misma categoría: “el sistema funciona aparentemente, los pods están `Ready`, los *logs* no muestran errores, pero los detectores están sordos”. La defensa es el test *end-to-end*: lanzar un ataque conocido (un `curl` con `?id=1' OR '1'='1`, un `kubect1 exec - sh`) y verificar que aparece el evento esperado en `Loki/Grafana` antes de dar el *stack* por terminado.

4.11 Estado tras Fase 4

Con las cuatro fases completas, un `vagrant up` seguido de tres `vagrant provision --provision-with` deja un laboratorio Kubernetes funcional, observable y con superficie de ataque definida, listo para *pen-test* simulado desde Kali. Los siguientes hitos no cubiertos en este documento (y reservados para iteraciones posteriores) incluyen el cableado de Alertmanager y un *bot* de Telegram para alertas *push*.

A Scripts

A continuación se listan todos los *scripts* del proyecto. Cada uno está incluido vía `\inputscript`, que lo lee directamente desde `../scripts/` al compilar el LaTeX. Esto significa que el contenido del PDF refleja el estado real del repositorio en el momento de la compilación — no hay copias estáticas que puedan desfasarse.

A.1 network-check.sh

Trigger de host (before :up, only_on: test-master). Compara la configuración activa de red-prueba en libvirt con el XML del repositorio y la recrea si difieren.

```
1 #!/bin/bash
2
3 NETWORK_NAME="red-prueba"
4 NETWORK_FILE="$(dirname "$0")/../../network/red-prueba.xml"
5
6 # Check if network exists in libvirt
7 if virsh -c qemu:///system net-info "$NETWORK_NAME" &>/dev/null; then
8     # Compare active config with local XML (ignore UUIDs, MACs, and whitespace differences)
9     ACTIVE=$(virsh -c qemu:///system net-dumpxml "$NETWORK_NAME" | grep -v '<uuid>' | grep
10         -v '<mac address=' | grep -v 'connections=' | sed 's/^ */')
11     LOCAL=$(cat "$NETWORK_FILE" | grep -v '<uuid>' | grep -v '<mac address=' | sed 's/^
12         */')
13
14     if [ "$ACTIVE" != "$LOCAL" ]; then
15         echo "[+] Red: $NETWORK_NAME cambió la configuración, recreándola..."
16         virsh -c qemu:///system net-destroy "$NETWORK_NAME" 2>/dev/null || true
17         virsh -c qemu:///system net-undefine "$NETWORK_NAME" 2>/dev/null || true
18         virsh -c qemu:///system net-define "$NETWORK_FILE"
19         virsh -c qemu:///system net-start "$NETWORK_NAME"
20     fi
21 else
22     # Network doesn't exist, create it
23     virsh -c qemu:///system net-define "$NETWORK_FILE"
24     virsh -c qemu:///system net-start "$NETWORK_NAME"
25 fi
26
27 virsh -c qemu:///system net-autostart "$NETWORK_NAME" 2>/dev/null || true
28
29 echo "[+] Red: $NETWORK_NAME está operativa"
```

A.2 create-disks.sh

Trigger de host (before :up, only_on: test-master). Crea los discos *qcow2 sparse* en `disks/` y configura las ACL para el usuario `libvirt-qemu`.

```
1 #!/bin/bash
2
3 SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
4 DISK_DIR="$SCRIPT_DIR/../../disks"
5
6 mkdir -p "$DISK_DIR"
7
8 # Grant libvirt-qemu traverse access to reach the disks directory
9 # (only adds ACL entries, harmless if already set)
10 setfacl -m u:libvirt-qemu:x "$HOME" 2>/dev/null || true
11 setfacl -m u:libvirt-qemu:rwX "$DISK_DIR" 2>/dev/null || true
12
```

```

13 declare -A SIZES=(
14   ["test-master"]=40G
15   ["test-worker1"]=20G
16   ["test-worker2"]=20G
17 )
18
19 for node in test-master test-worker1 test-worker2; do
20   DISK="$DISK_DIR/${node}-data.qcow2"
21   if [ -f "$DISK" ]; then
22     echo "[!] El disco $DISK ya existe, omitiendo..."
23   else
24     qemu-img create -f qcow2 "$DISK" "${SIZES[$node]}"
25     setfacl -m u:libvirt-qemu:rw "$DISK"
26     echo "[+] Creado $DISK (${SIZES[$node]})"
27   fi
28 done

```

A.3 config.sh

Provisioner shell que se ejecuta en las tres VMs en cada up. Instala paquetes base (vim, python3, python3-pip, python3-venv, git).

```

1 #!/bin/bash
2
3 export DEBIAN_FRONTEND=noninteractive
4 set -eou pipefail
5
6 echo "[+] Instalando paquetes..."
7 apt-get update -qq
8 apt-get install vim python3 python3-pip python3-venv git -y >/dev/null

```

A.4 ssh-setup.sh

Provisioner shell que se ejecuta en las tres VMs. Crea el usuario personalizado, copia la clave pública de Vagrant a `authorized_keys` e instala la clave compartida del lab para SSH entre nodos.

```

1 #!/bin/bash
2
3 export DEBIAN_FRONTEND=noninteractive
4 set -eou pipefail
5
6 USERNAME="$1"
7
8 # Shared lab keypair for inter-node SSH (master <-> workers)
9 SSH_PRIVATE_KEY="-----BEGIN OPENSSH PRIVATE KEY-----
10 b3B1bnNzaC1rZXktdjEAAAABG5vbmUAAAABbm9uZQAAAAAAAAABAAAMwAAAAAtzc2gtZW
11 QyNTUxOQAAACA979XNR/W5ElhvK2gLBm8aDfcQRoiiln+/8Bavqb+daQAAAJiwBbnRsAW5
12 OQAAAtzc2gtZWQyNTUxOQAAACA979XNR/W5ElhvK2gLBm8aDfcQRoiiln+/8Bavqb+daQ
13 AAABBBgSKL6zh/DDhOC4bMFo4TcJnija0qbSt+vKQiVkpCxT3v1c1H9bkSWG8raAsGbxoN
14 9xBGiKKWf7/wFq+pv51pAAAAFWxhYkByYXJhbGVsby1wcm95ZWNoYw==
15 -----END OPENSSH PRIVATE KEY-----"
16 SSH_PUBLIC_KEY="ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAID3v1c1H9bkSWG8raAsGbxoN9xBGiKKWf7/
17 wFq+pv51p lab@paralelo-proyecto"
18
19 echo "[+] Configurando usuario: $USERNAME"
20
21 if ! id "$USERNAME" &>/dev/null; then
22   useradd -m -s /bin/bash "$USERNAME"
23   echo "$USERNAME:$USERNAME" | chpasswd
24   usermod -aG sudo "$USERNAME"
25   echo "$USERNAME ALL=(ALL) NOPASSWD: ALL" >/etc/sudoers.d/$USERNAME
26 fi
27
28 USER_HOME="/home/$USERNAME"
29 mkdir -p "$USER_HOME/.ssh"
30
31 # Host -> VM SSH (copy vagrant's key)
32 cp /home/vagrant/.ssh/authorized_keys "$USER_HOME/.ssh/authorized_keys"
33
34 # Inter-node SSH (shared lab keypair)

```

```

34 echo "$SSH_PRIVATE_KEY" >"$USER_HOME/.ssh/id_ed25519"
35 echo "$SSH_PUBLIC_KEY" >"$USER_HOME/.ssh/id_ed25519.pub"
36 if ! grep -qF "$SSH_PUBLIC_KEY" "$USER_HOME/.ssh/authorized_keys" 2>/dev/null; then
37     echo "$SSH_PUBLIC_KEY" >>"$USER_HOME/.ssh/authorized_keys"
38 fi
39
40 # Skip host key verification for lab subnet
41 cat >"$USER_HOME/.ssh/config" <<EOF
42 Host 10.0.1.*
43     StrictHostKeyChecking no
44     UserKnownHostsFile /dev/null
45 EOF
46
47 chown -R "$USERNAME:$USERNAME" "$USER_HOME/.ssh"
48 chmod 700 "$USER_HOME/.ssh"
49 chmod 600 "$USER_HOME/.ssh/id_ed25519"
50 chmod 600 "$USER_HOME/.ssh/authorized_keys"
51 chmod 644 "$USER_HOME/.ssh/id_ed25519.pub"
52 chmod 644 "$USER_HOME/.ssh/config"

```

A.5 ssh-config.sh

Trigger de host (after :up, only_on: test-worker2). Escribe entradas SSH en ~/.ssh/config del host con los usuarios personalizados, envueltas entre marcadores para idempotencia.

```

1  #!/bin/bash
2
3  SSH_CONFIG="$HOME/.ssh/config"
4  MARKER_START="# BEGIN paralelo-proyecto"
5  MARKER_END="# END paralelo-proyecto"
6
7  BLOCK=$(vagrant ssh-config 2>/dev/null)
8
9  if [ -z "$BLOCK" ]; then
10     echo "Failed to get vagrant ssh-config"
11     exit 1
12 fi
13
14 # Replace User vagrant with custom users per host
15 BLOCK=$(echo "$BLOCK" | awk '
16     /^Host test-master/ { user="master" }
17     /^Host test-worker1/ { user="worker1" }
18     /^Host test-worker2/ { user="worker2" }
19     /User vagrant/ && user { gsub(/User vagrant/, "User " user); user="" }
20     { print }
21 ')
22
23 mkdir -p "$HOME/.ssh"
24
25 if [ -f "$SSH_CONFIG" ]; then
26     CLEANED=$(awk -v start="$MARKER_START" -v end="$MARKER_END" \
27         '$0 == start {skip=1; next} $0 == end {skip=0; next} !skip' "$SSH_CONFIG")
28     echo "$CLEANED" >"$SSH_CONFIG"
29 else
30     touch "$SSH_CONFIG"
31     chmod 600 "$SSH_CONFIG"
32 fi
33
34 printf '%s\n%s\n%s\n' "$MARKER_START" "$BLOCK" "$MARKER_END" >>"$SSH_CONFIG"
35
36 echo "[+] Configuración SSH actualizada para paralelo-proyecto"

```

A.6 attach-disks.sh

Trigger de host (after :up, only_on: test-worker2). Conecta los qcow2 como vdb con virsh attach-disk --persistent, formatea ext4 si está vacío, monta en /mnt/data y persiste vía /etc/fstab.

```

1  #!/bin/bash
2
3  VIRSH="virsh -c qemu:///system"

```

```

4 SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
5 PROJECT_DIR="$SCRIPT_DIR/.."
6 DISK_DIR="$PROJECT_DIR/disks"
7
8 for node in test-master test-worker1 test-worker2; do
9     DISK="$(realpath "$DISK_DIR/${node}-data.qcow2")"
10    ID_FILE="$PROJECT_DIR/.vagrant/machines/${node}/libvirt/id"
11
12    if [ ! -f "$ID_FILE" ]; then
13        echo "[!] ERROR: No se ha encontrado ID de dominio de libvirt para $node -- omitiendo"
14        continue
15    fi
16
17    DOMAIN=$(cat "$ID_FILE")
18
19    # Check if vdb is already attached
20    if $VIRSH domblklist "$DOMAIN" | grep -q "vdb"; then
21        echo "[!] $node: vdb ya se encuentra conectado"
22    else
23        echo "[+] $node: conectado $DISK como vdb"
24        $VIRSH attach-disk "$DOMAIN" "$DISK" vdb \
25            --driver qemu --subdriver qcow2 --persistent
26    fi
27
28    # Format and mount inside the VM via SSH
29    ssh -o StrictHostKeyChecking=no "$node" bash -s <<'REMOTE'
30    set -e
31
32    # Wait briefly for the device to appear
33    for i in $(seq 1 10); do
34        [ -b /dev/vdb ] && break
35        sleep 1
36    done
37
38    if [ ! -b /dev/vdb ]; then
39        echo "[!] ERROR: /dev/vdb no encontrado"
40        exit 1
41    fi
42
43    # Format only if no filesystem exists
44    if ! sudo blkid /dev/vdb &>/dev/null; then
45        echo "[+] Formateando /dev/vdb como ext4"
46        sudo mkfs.ext4 -q /dev/vdb
47    else
48        echo "[!] /dev/vdb already ya tiene un sistema de ficheros"
49    fi
50
51    # Create mount point
52    sudo mkdir -p /mnt/data
53
54    # Add fstab entry if not present
55    if ! grep -q '/dev/vdb' /etc/fstab; then
56        echo '/dev/vdb /mnt/data ext4 defaults,nofail 0 2' | sudo tee -a /etc/fstab >/dev/
57        null
58        echo "[!] Entrada añadida en /etc/fstab"
59    fi
60
61    # Mount if not already mounted
62    if ! mountpoint -q /mnt/data; then
63        sudo mount /mnt/data
64        echo "[!] Montaje realizado en /mnt/data"
65    else
66        echo "[!] /mnt/data ya se encuentra montado"
67    fi
68
69    REMOTE
70
71    echo "[*] $node: terminado"
72    echo
73done

```

A.7 setup-kubespray.sh

Provisioner run: "never" en test-master. Despliega Kubernetes v1.30.4 con Kubespray v2.26.0 (Flannel CNI, containerd) y crea los tres *namespaces*.

Invocación: `vagrant provision test-master --provision-with setup-kubespray`

```
1 #!/bin/bash
2 #
3 # Runs inside test-master as a Vagrant shell provisioner (privileged=false).
4 # Re-execs as the 'master' user so everything lands under /home/master/.
5 # Deploys a 3-node Kubernetes cluster with kubespray v2.26.0.
6 # Assumes config.sh + ssh-setup.sh have already run on all nodes
7 # (python3, git, custom users, shared lab keypair, passwordless sudo).
8 #
9 set -euo pipefail
10
11 # Re-exec as master if we're not already (Vagrant runs this as 'vagrant').
12 # The uploaded script in /tmp is mode 700 owned by vagrant, so master can't
13 # read it until we widen the read bit (vagrant owns it, no sudo needed).
14 if [ "$(id -un)" != "master" ]; then
15     chmod a+r "$0"
16     exec sudo -iu master bash "$0" "$@"
17 fi
18
19 INVENTORY_NAME="cluster-k8-paralelo"
20 KUBESPRAY_TAG="v2.26.0"
21 KUBESPRAY_DIR="$HOME/kubespray"
22 VENV_DIR="$HOME/kubespray-venv"
23 INV_DIR="$KUBESPRAY_DIR/inventory/$INVENTORY_NAME"
24
25 #####
26 # Phase 1: Clone kubespray
27 #####
28 echo "[!] Fase 1: Clonando kubespray ${KUBESPRAY_TAG}"
29 if [ -d "$KUBESPRAY_DIR/.git" ]; then
30     git -C "$KUBESPRAY_DIR" fetch --tags
31     git -C "$KUBESPRAY_DIR" checkout "$KUBESPRAY_TAG"
32 else
33     git clone --branch "$KUBESPRAY_TAG" --depth 1 \
34         https://github.com/kubernetes-sigs/kubespray.git "$KUBESPRAY_DIR"
35 fi
36
37 #####
38 # Phase 2: Python virtualenv + requirements
39 #####
40 echo "[!] Fase 2: Montando entorno virtual en Python"
41 [ -d "$VENV_DIR" ] || python3 -m venv "$VENV_DIR"
42 # shellcheck disable=SC1091
43 source "$VENV_DIR/bin/activate"
44 pip install -q -U pip
45 pip install -q -r "$KUBESPRAY_DIR/requirements.txt"
46
47 #####
48 # Phase 3: Generate inventory (hosts.yml with ansible_user per host)
49 #####
50 echo "[!] Fase 3: Generando inventario en $INV_DIR"
51 mkdir -p "$INV_DIR/group_vars"
52
53 cat >"$INV_DIR/hosts.yml" <<'INVENTORY'
54 all:
55     hosts:
56         master:
57             ansible_host: 10.0.1.10
58             ansible_user: master
59             ip: 10.0.1.10
60             access_ip: 10.0.1.10
61         worker1:
62             ansible_host: 10.0.1.11
63             ansible_user: worker1
64             ip: 10.0.1.11
65             access_ip: 10.0.1.11
66         worker2:
67             ansible_host: 10.0.1.12
```

```

68     ansible_user: worker2
69     ip: 10.0.1.12
70     access_ip: 10.0.1.12
71 children:
72     kube_control_plane:
73         hosts:
74             master:
75     kube_node:
76         hosts:
77             worker1:
78             worker2:
79     etcd:
80         hosts:
81             master:
82     k8s_cluster:
83         children:
84             kube_control_plane:
85             kube_node:
86     calico_rr:
87         hosts: {}
88 INVENTORY
89
90 #####
91 # Phase 4: Configure group_vars (copy sample, switch CNI to flannel)
92 #####
93 echo "[!] Fase 4: Configurando group_vars"
94 cp -r "$KUBESPRAY_DIR/inventory/sample/group_vars/" "$INV_DIR/group_vars/"
95
96 sed -i 's/^kube_network_plugin:.*\/kube_network_plugin: flannel/' \
97     "$INV_DIR/group_vars/k8s_cluster/k8s-cluster.yml"
98
99 #####
100 # Phase 5: Run ansible-playbook
101 #####
102 echo "[!] Fase 5: Ejecutando ansible-playbook"
103 cd "$KUBESPRAY_DIR"
104 ansible-playbook -i "inventory/$INVENTORY_NAME/hosts.yml" --become cluster.yml
105
106 #####
107 # Phase 6: kubeconfig for local kubectl
108 #####
109 echo "[!] Fase 6: Instalando kubeconfig para el usuario master"
110 mkdir -p "$HOME/.kube"
111 sudo cp /etc/kubernetes/admin.conf "$HOME/.kube/config"
112 sudo chown "$(id -u):$(id -g)" "$HOME/.kube/config"
113
114 echo '[+] NODOS'
115 kubectl get nodes
116 echo '[+] PODS'
117 kubectl get pods -A
118
119 echo "[+] Despliegue de Kubespray exitoso"
120
121 #####
122 # Phase 7: creando namespaces
123 #####
124 echo "[!] Fase 7: Creando namespaces (vulnerable, monitoring, security)"
125 kubectl create namespace vulnerable
126 kubectl create namespace monitoring
127 kubectl create namespace security
128
129 #echo "[+] Fase 7: Instalando helm para el usuario master"
130 #####
131 # Phase 8: installing helm
132 #####
133 #echo "[+] Fase 7: Instalando helm para el usuario master"
134
135 #curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-
136     helm-4
137 #chmod 700 get_helm.sh
138 #bash get_helm.sh

```

A.8 setup-lab.sh

Provisioner run: "never" en test-master. Despliega el *stack* de monitorización (Helm 4 si falta, repos, chown de hostPaths, PV/PVC, Grafana, Prometheus, Loki, Promtail).

Invocación: `vagrant provision test-master --provision-with setup-lab`

```
1 #!/bin/bash
2 #
3 # Runs inside test-master as a Vagrant shell provisioner (privileged=false).
4 # Re-execs as the 'master' user so helm/kubectl hablan contra el kubeconfig
5 # de master (~/.kube/config).
6 #
7 # Despliega:
8 #   - Helm 4 (si no está)
9 #   - Grafana standalone con datasources Prometheus+Loki provisionados
10 #     declarativamente desde values/grafana.yaml (ya no hay POST /api/datasources)
11 #   - Prometheus standalone (chart prometheus-community/prometheus)
12 #   - Loki standalone en modo SingleBinary (chart grafana/loki)
13 #   - Promtail DaemonSet enviando logs a Loki por DNS interno
14 #
15 # Idempotente: re-ejecutable sin romper nada (helm upgrade --install,
16 # kubectl apply, chown -- todos idempotentes por diseño).
17 #
18 # Prerequisitos:
19 #   - Cluster k8s levantado con setup-kubespray.sh (Fase 7 ya corrida -- el
20 #     namespace 'monitoring' tiene que existir).
21 #   - /vagrant sincronizado con el repo (vagrant rsync si hace falta).
22 #
23 set -euo pipefail
24
25 # Re-exec como master si no estamos ya (Vagrant arranca como 'vagrant').
26 # El script sube a /tmp con modo 700 de vagrant, por eso le damos a+r antes
27 # de cambiar a master, que si no no puede ni leerlo.
28 if [ "$(id -un)" != "master" ]; then
29     chmod a+r "$0"
30     exec sudo -iu master bash "$0" "$@"
31 fi
32
33 MONITORING_NS="monitoring"
34 GRAFANA_NODE_IP="10.0.1.12"      # worker2 -- mismo nodo que aloja los hostPath
35 GRAFANA_NODEPORT="30711"
36 PROMETHEUS_NODEPORT="30090"
37
38 #####
39 # Precondición: el cluster y el namespace monitoring tienen que existir
40 #####
41 if ! kubectl get namespace "$MONITORING_NS" >/dev/null 2>&1; then
42     echo "[x] El namespace '$MONITORING_NS' no existe."
43     echo "Ejecuta antes: vagrant provision test-master --provision-with setup-
44         kubespray"
45     exit 1
46 fi
47
48 #####
49 # Fase 1: Instalar Helm 4 (si no está)
50 #####
51 echo "[!] Fase 1: Instalando Helm 4"
52 if command -v helm >/dev/null 2>&1; then
53     echo "Ya instalado: $(helm version --short)"
54 else
55     curl -fsSL -o /tmp/get_helm.sh https://raw.githubusercontent.com/helm/helm/main/
56         scripts/get_helm-4
57     chmod 700 /tmp/get_helm.sh
58     bash /tmp/get_helm.sh
59     rm -f /tmp/get_helm.sh
60 fi
61
62 #####
63 # Fase 2: Repos de Helm
64 #####
65 echo "[!] Fase 2: Añadiendo repos de Helm"
66 # 'helm repo add' falla si el repo ya existe; con '|| true' lo hacemos idempotente.
67 helm repo add grafana https://grafana.github.io/helm-charts >/
```

```

dev/null 2>&1 || true
66 helm repo add prometheus-community https://prometheus-community.github.io/helm-charts >/
dev/null 2>&1 || true
67 helm repo update
68
69 #####
70 # Fase 3: Pre-chown de hostPaths en worker2
71 #####
72 # fsGroup en el PodSpec NO recursiza sobre hostPath -- por eso el chown manual
73 # es obligatorio. UID 472 = usuario 'grafana' en la imagen oficial;
74 # UID 65534 = 'nobody', usuario del server de Prometheus;
75 # UID 10001 = usuario que el chart de Loki configura en su podSecurityContext.
76 echo "[!] Fase 3: Preparando hostPaths en worker2"
77 ssh -o StrictHostKeyChecking=accept-new worker2@10.0.1.12 'bash -s' <<'REMOTE'
78 set -euo pipefail
79 sudo mkdir -p /mnt/data/grafana /mnt/data/prometheus /mnt/data/loki
80 sudo chown 472:472 /mnt/data/grafana
81 sudo chown 65534:65534 /mnt/data/prometheus
82 sudo chown 10001:10001 /mnt/data/loki
83 REMOTE
84
85 #####
86 # Fase 4: PV/PVC
87 #####
88 echo "[!] Fase 4: Aplicando PV/PVC"
89 kubectl apply -f /vagrant/manifests/storage/grafana-pv.yaml
90 kubectl apply -f /vagrant/manifests/storage/prometheus-pv.yaml
91 kubectl apply -f /vagrant/manifests/storage/loki-pv.yaml
92
93 #####
94 # Fase 5: Grafana standalone
95 #####
96 # --wait bloquea hasta que todos los recursos del release estén Ready, así la
97 # fase 7 (llamada a la API de Grafana) tiene garantía de que el pod existe.
98 echo "[!] Fase 5: Desplegando Grafana"
99 helm upgrade --install grafana grafana/grafana \
100 --namespace "$MONITORING_NS" \
101 -f /vagrant/values/grafana.yaml \
102 --wait --timeout 5m
103
104 #####
105 # Fase 6: Prometheus standalone
106 #####
107 echo "[!] Fase 6: Desplegando Prometheus"
108 helm upgrade --install prometheus prometheus-community/prometheus \
109 --namespace "$MONITORING_NS" \
110 -f /vagrant/values/prometheus.yaml \
111 --wait --timeout 5m
112
113 #####
114 # Fase 7: Loki standalone (SingleBinary, hostPath en worker2)
115 #####
116 # El cableado del datasource Lokien Grafana se hace declarativamente vía
117 # values/grafana.yaml (sección 'datasources'), no aquí. El orden importa
118 # poco: Grafana se instaló en Fase 5 con la config del datasource ya
119 # cargada; mientras Loki no esté Ready, las queries fallan (esperado).
120 # Cuando este helm install termine, las queries empiezan a funcionar.
121 echo "[!] Fase 7: Desplegando Loki"
122 helm upgrade --install loki grafana/loki \
123 --namespace "$MONITORING_NS" \
124 -f /vagrant/values/loki.yaml \
125 --wait --timeout 5m
126
127 #####
128 # Fase 8: Promtail DaemonSet (workers, push a Loki interno)
129 #####
130 # Promtail corre solo en worker1+worker2 -- el override 'tolerations: []'
131 # en values/promtail.yaml impide que tolere el taint del control-plane,
132 # así el DaemonSet evita master automáticamente.
133 echo "[!] Fase 8: Desplegando Promtail"
134 helm upgrade --install promtail grafana/promtail \
135 --namespace "$MONITORING_NS" \
136 -f /vagrant/values/promtail.yaml \

```



```

137 --wait --timeout 5m
138
139 #####
140 # Fase 9: Resumen
141 #####
142 # Leemos la contraseña del admin desde el Secret que el chart gestiona --
143 # así el output siempre refleja la pass real, no la del values en el
144 # momento de la primera instalación (helm no rota Secrets en upgrades).
145 admin_pass=$(kubectl get secret -n "$MONITORING_NS" grafana \
146 -o jsonpath="{.data.admin-password}" | base64 -d)
147
148 echo
149 echo "[+] Lab de monitoring desplegado."
150 echo "    - Grafana:      http://${GRAFANA_NODE_IP}:${GRAFANA_NODEPORT}    (admin / ${
151     admin_pass})"
152 echo "    - Prometheus: http://${GRAFANA_NODE_IP}:${PROMETHEUS_NODEPORT}"
153 echo "    - Loki:         solo interno (http://loki.${MONITORING_NS}.svc.cluster.local
154     :3100)"
155 echo "    - Datasources cableados vía Grafana provisioning (values/grafana.yaml)"
156 echo
157 kubectl get pods -n "$MONITORING_NS"

```

A.9 setup-security.sh

Provisioner run: "never" en test-master. Despliega Falco, Suricata y los tres CMS vulnerables (DVWA, WebGoat, Juice Shop).

Invocación: `vagrant provision test-master --provision-with setup-security`

```

1 #!/bin/bash
2 #
3 # Runs inside test-master as a Vagrant shell provisioner (privileged=false).
4 # Re-execs as the 'master' user -- mismo pattern que setup-lab.sh -- para que
5 # helm/kubectl usen el kubeconfig real del cluster (~/.kube/config).
6 #
7 # Despliega el stack de seguridad COMPLETO:
8 #   - Falco standalone (chart upstream falcosecurity/falco) -- DaemonSet
9 #     en workers, driver modern_ebpf, output JSON a stdout
10 #   - Suricata standalone (chart custom charts/suricata) -- DaemonSet en
11 #     worker1 con hostNetwork, sidecar tail eve.json → stdout
12 #   - DVWA (chart custom charts/dvwa) -- Deployment 1 replica en worker1
13 #     con MySQL embebido (sin persistencia -- ver charts/dvwa/values.yaml)
14 #
15 # Pipeline final de eventos:
16 #   Falco/Suricata stdout → Promtail → Loki → Grafana
17 #   Tráfico Kali → eth0 worker1 → Suricata af-packet → eve.json → ↑
18 #   Tráfico Kali → DVWA :30430 → Apache logs (ya en stdout)      → ↑
19 #
20 # Precondición fuerte: el stack de monitoring (Loki + Promtail) tiene que
21 # estar corriendo. Sin Promtail, los eventos se generan pero nunca llegan
22 # a Loki -- el script aborta antes de instalar nada para evitar dejar
23 # detectores "huérfanos".
24 #
25 # Idempotente: re-ejecutable sin romper nada (helm upgrade --install,
26 # kubectl apply con dry-run de namespace, mkdir -p, chown idempotente).
27 #
28 # Prerequisitos:
29 #   - Cluster k8s levantado con setup-kubespray.sh
30 #   - Stack de monitoring desplegado con setup-lab.sh
31 #   - /vagrant sincronizado con el repo (vagrant rsync si hace falta)
32 #
33 set -euo pipefail
34
35 # Re-exec como master si no estamos ya. Vagrant arranca como 'vagrant',
36 # que no tiene ~/.kube/config. El chmod a+r es necesario porque el script
37 # llega a /tmp con modo 700 y el user master no podría leerlo.
38 if [ "$(id -un)" != "master" ]; then
39     chmod a+r "$0"
40     exec sudo -iu master bash "$0" "$@"
41 fi
42
43 MONITORING_NS="monitoring"

```

```

44 SECURITY_NS="security"
45 VULNERABLE_NS="vulnerable"
46
47 DVWA_NODE_IP="10.0.1.11" # worker1 -- donde corre DVWA + Suricata
48 DVWA_NODEPORT="30430"
49
50 #####
51 # Fase 1: Precondiciones -- el stack de monitoring tiene que estar listo
52 #####
53 # Detectores sin Promtail = eventos perdidos. Verificamos que el namespace
54 # exista y que el DaemonSet de Promtail tenga al menos 1 pod Ready.
55 echo "[!] Fase 1: Verificando precondiciones del stack de monitoring"
56
57 if ! kubectl get namespace "$MONITORING_NS" >/dev/null 2>&1; then
58     echo "[x] El namespace '$MONITORING_NS' no existe."
59     echo "     Ejecuta antes: vagrant provision test-master --provision-with setup-lab"
60     exit 1
61 fi
62
63 promtail_ready=$(kubectl get ds -n "$MONITORING_NS" promtail \
64 -o jsonpath='{.status.numberReady}' 2>/dev/null || echo "0")
65 if [ "$promtail_ready" -lt 1 ]; then
66     echo "[x] Promtail no tiene pods Ready en el ns '$MONITORING_NS'."
67     echo "     Sin Promtail los eventos no llegan a Loki."
68     echo "     Ejecuta antes: vagrant provision test-master --provision-with setup-lab"
69     exit 1
70 fi
71 echo "     Promtail Ready en $promtail_ready nodo(s) -- OK"
72
73 #####
74 # Fase 2: Repos de Helm
75 #####
76 # Solo Falco viene de un repo upstream. Suricata y DVWA son charts custom
77 # en charts/ -- se instalan apuntando a la ruta local, sin repo.
78 #
79 # OJO con el tipo del nombre: 'falcosecurity' (sin 'n'). Lección del vault:
80 # 'helm repo add' no valida nombres -- un tipo (falconsecurity) se añade
81 # silenciosamente y 'helm install' falla con "repo not found" sin pista
82 # de tipo. El 'helm repo update' posterior valida que resuelve.
83 echo "[!] Fase 2: Añadiendo repo Helm de Falco"
84 helm repo add falcosecurity https://falcosecurity.github.io/charts >/dev/null 2>&1 ||
85     true
86 helm repo update
87
88 #####
89 # Fase 3: Namespaces
90 #####
91 # kubectl create con --dry-run + apply -f - es el patrón idempotente
92 # canónico: si el ns ya existe, apply no hace nada; si no, lo crea.
93 # Evita el '| true' que enmascararía errores distintos a AlreadyExists.
94 #
95 # Dos namespaces: 'security' (detectores) + 'vulnerable' (DVWA).
96 # Separación deliberada -- un compromiso de DVWA no debe ver Falco/Suricata.
97 echo "[!] Fase 3: Asegurando namespaces '$SECURITY_NS' y '$VULNERABLE_NS'"
98 for ns in "$SECURITY_NS" "$VULNERABLE_NS"; do
99     kubectl create namespace "$ns" --dry-run=client -o yaml | kubectl apply -f -
100 done
101
102 #####
103 # Fase 4: Falco standalone
104 #####
105 # --wait bloquea hasta que el DaemonSet tenga todos los pods Ready
106 # (initContainer falcoctl-artifact-install + main + sidecar follow).
107 # En kernel 5.15 con modern_ebpf el arranque tarda ~30-60s.
108 echo "[!] Fase 4: Desplegando Falco"
109 helm upgrade --install falco falcosecurity/falco \
110     --namespace "$SECURITY_NS" \
111     -f /vagrant/values/falco.yaml \
112     --wait --timeout 5m
113
114 #####
115 # Fase 5: Suricata (chart custom)
116 #####

```

```

116 # helm install apuntando a la RUTA LOCAL del chart, no a un repo. La ruta
117 # /vagrant/charts/suricata es el directorio sincronizado con el repo del host.
118 #
119 # --wait: para un DaemonSet con replicas=1 (por nodeSelector) significa
120 # esperar a que ese único pod esté Ready. El initContainer suricata-update
121 # tarda ~30s descargando ET Open la primera vez (~30k reglas).
122 #
123 # NB: NO hay fase de PV/PVC para DVWA. La imagen vulnerables/web-dvwa trae
124 # la BD MariaDB horneada en la imagen y su entrypoint NO ejecuta
125 # mysql_install_db -- montar un PVC encima de /var/lib/mysql shadowea la BD
126 # pre-cocinada y mysqld muere con "Table 'mysql.user' doesn't exist".
127 # Decisión: emptyDir implícito (sin persistencia). Reset de DVWA es parte
128 # del workflow normal del lab. Ver charts/dvwa/values.yaml.
129 echo "[!] Fase 5: Desplegando Suricata"
130 helm upgrade --install suricata /vagrant/charts/suricata \
131   --namespace "$SECURITY_NS" \
132   -f /vagrant/values/suricata.yaml \
133   --wait --timeout 5m
134
135 #####
136 # Fase 6: DVWA (chart custom)
137 #####
138 # Mismo patrón que Suricata: chart local, values overrides aparte.
139 # Sin PVC -- el datadir de MariaDB vive en el filesystem efímero del pod.
140 # Cada restart obliga a re-hacer setup desde /setup.php (~5s manual).
141 echo "[!] Fase 6: Desplegando DVWA"
142 helm upgrade --install dvwa /vagrant/charts/dvwa \
143   --namespace "$VULNERABLE_NS" \
144   -f /vagrant/values/dvwa.yaml \
145   --wait --timeout 5m
146
147 #####
148 # Fase 7: WebGoat (chart custom)
149 #####
150 # Java + Spring Boot autocontenido. Tarda ~60-90s en pasar Ready la primera
151 # vez (classpath scan + H2 init). El chart pone initialDelaySeconds=120 en
152 # el liveness probe para que kubelet no lo mate prematuramente.
153 #
154 # JAVA_TOOL_OPTIONS=-Xmx384m acota el heap para que no rebase el limit
155 # de 512Mi del container -- sin ese cap, Spring Boot crece hasta donde le
156 # deja el host y dispara OOMKilled (exit 137) en worker1 (apretado de RAM).
157 #
158 # --timeout 8m: holgura para el cold-start. Pull de la imagen (~700MB) +
159 # arranque JVM puede llegar a 5min en redes lentas la primera vez.
160 echo "[!] Fase 7: Desplegando WebGoat"
161 helm upgrade --install webgoat /vagrant/charts/webgoat \
162   --namespace "$VULNERABLE_NS" \
163   -f /vagrant/values/webgoat.yaml \
164   --wait --timeout 8m
165
166 #####
167 # Fase 8: Juice Shop (chart custom)
168 #####
169 # Node.js + Express + sqlite efímera. Mucho más liviano que WebGoat --
170 # Ready en ~15s. Sin env vars críticas, defaults de la imagen sirven.
171 echo "[!] Fase 8: Desplegando Juice Shop"
172 helm upgrade --install juiceshop /vagrant/charts/juiceshop \
173   --namespace "$VULNERABLE_NS" \
174   -f /vagrant/values/juiceshop.yaml \
175   --wait --timeout 5m
176
177 #####
178 # Fase 9: Resumen
179 #####
180 WEBGOAT_NODEPORT="30380"
181 JUICESHOP_NODEPORT="31592"
182
183 echo
184 echo "[+] Stack de seguridad desplegado."
185 echo "  - Falco      → ns '$SECURITY_NS'      (workers, master excluido)"
186 echo "  - Suricata    → ns '$SECURITY_NS'      (solo worker1, hostNetwork)"
187 echo "  - DVWA        → ns '$VULNERABLE_NS'    (worker1, MySQL efímero)"
188 echo "  - WebGoat     → ns '$VULNERABLE_NS'    (worker1, H2 efímera)"

```

```

189 echo "      - Juice Shop → ns '$VULNERABLE_NS' (worker1, sqlite efimera)"
190 echo
191 echo "      Endpoints (acceso desde host o Kali):"
192 echo "          DVWA:      http://${DVWA_NODE_IP}:${DVWA_NODEPORT}/login.php"
193 echo "          Setup inicial: visitar /setup.php → 'Create / Reset Database'"
194 echo "          Credenciales: admin / password"
195 echo "          WebGoat:     http://${DVWA_NODE_IP}:${WEBGOAT_NODEPORT}/WebGoat (case -
sensitive)"
196 echo "          Crear cuenta nueva en el form de registro la primera vez"
197 echo "          Juice Shop:  http://${DVWA_NODE_IP}:${JUICESHOP_NODEPORT}/"
198 echo "          Sin login inicial requerido -- explora /#/score-board para retos"
199 echo
200 kubectl get pods -n "$SECURITY_NS" -o wide
201 kubectl get pods -n "$VULNERABLE_NS" -o wide
202 echo
203 echo "      Triggers de prueba:"
204 echo "          # Falco -- shell en container:"
205 echo "          kubectl exec -n monitoring -it \$(kubectl get pod -n monitoring -l app.
kubernetes.io/name=grafana -o name | head -1) -- /bin/sh -c 'id; exit'"
206 echo "          # Suricata -- petición a cada CMS desde el master:"
207 echo "          curl -s -o /dev/null -w 'DVWA      %{http_code}\n' http://${DVWA_NODE_IP}:$
${DVWA_NODEPORT}/login.php"
208 echo "          curl -s -o /dev/null -w 'WebGoat    %{http_code}\n' http://${DVWA_NODE_IP}:$
${WEBGOAT_NODEPORT}/WebGoat/login.mvc"
209 echo "          curl -s -o /dev/null -w 'Juice Shop %{http_code}\n' http://${DVWA_NODE_IP}:$
${JUICESHOP_NODEPORT}/"
210 echo
211 echo "      Queries LogQL en Grafana → Explore → Loki:"
212 echo "          (Promtail indexa con label corto 'app=', NO 'app.kubernetes.io/name')"
213 echo "          Falco:      {namespace=\"security\", app=\"falco\"} | json | rule != \"\""
214 echo "          Suricata: {namespace=\"security\", app=\"suricata\"} | json | event_type = \
\"alert\""

```